Universidad San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Curso: Sistemas de Bases de Datos 2
Sección: A
Ingeniero: Daniel Alberto Gonzáles Gonzáles



MANUAL TÉCNICO

Proyecto Final - Admisión Ingeniería USAC

Estudiante: Herberth Julian Reyes Pacajoj
Registro Académico: 202230236

Quetzaltenango, octubre 2026

Contenido

1. Introducción y alcance 4
    1.1 Contexto 4
    1.2 Alcance 4
    1.3 Datos de origen 4
    1.4 Conceptos clave 5
2. Arquitectura y tecnologías 7
    2.1 Visión general 7
    2.2 Tecnologías 8
        Por qué estas tecnologías 8
    2.3 Estructura del repositorio 9
    2.4 Principios de diseño 9
    2.5 Decisiones técnicas relevantes 10
3. Instalación y configuración 11
    3.1 Requisitos 11
    3.2 Puesta en marcha 11
    3.3 Variables de entorno 11
    3.4 Servicios y puertos 12
    3.5 Comandos disponibles 12
    3.6 Solución de problemas 13
4. Proceso ETL 14
    4.1 Visión general del flujo 14
    4.2 Estructura de los archivos de origen 15
    4.3 Validación y causas de rechazo 15
    4.4 Transformación 16
        4.4.1 Identificador del aspirante 16
        4.4.2 Carrera 17
        4.4.3 Geografía e institución 17
        4.4.4 Edad 17
    4.5 Duplicados y anomalías 18
    4.6 Aprobación y primer intento 18
        4.6.1 Vigencia de las aprobaciones 19
        4.6.2 Indicador principal y complementario 19
        4.6.3 Estado de cada postulación 20
        4.6.4 Primer intento 20
    4.7 Carga en las bases de datos 20
        4.7.1 MongoDB 20
        4.7.2 Neo4j 21
    4.8 Reporte de la carga y resultados 21
    4.9 Recarga, reconstrucción y borrado 22
5. Modelo de MongoDB 23
    5.1 Colecciones 23

Embeber o referenciar..... 24
5.2 Colección evaluaciones..... 24
5.3 Colección postulaciones..... 25
5.4 Colección aspirantes..... 26
5.5 Colecciones resumen_carrera, cargas y rechazados..... 27
5.6 Índices..... 27
6. Modelo de Neo4j..... 29
6.1 Conceptos básicos..... 29
6.2 Esquema..... 29
6.3 Decisiones de diseño..... 30
6.4 Restricciones e índices..... 31
6.5 Carga del grafo..... 31
8. Consultas..... 32
8.1 Criterios de diseño..... 32
8.2 Consultas de MongoDB..... 32
8.3 Consultas de Cypher..... 33
9. Análisis de resultados..... 34
9.1 Cómo leer los resultados..... 34
9.2 Panorama general..... 34
9.3 Evolución por año..... 35
9.4 Por materia..... 37
9.5 Reintentos y perseverancia..... 37
9.6 Tipo de institución..... 38
9.7 Edad y sexo..... 39
9.8 Carreras y geografía..... 39
9.9 Relaciones entre carreras (Neo4j)..... 41
9.10 Calidad de los datos y su efecto en el análisis..... 41
11. Conclusiones..... 43
11.1 Sobre los datos..... 43
11.2 Sobre MongoDB y Neo4j trabajando juntos..... 43
11.3 Limitaciones..... 44
Anexo A. Consultas de MongoDB..... 45
A.1 Consultas 1 a 10..... 45
Consulta 1 · Aspirantes por tipo de institución educativa..... 45
Consulta 2 · Cantidad de evaluaciones aprobadas por materia..... 45
Consulta 3 · Aspirantes aprobados por carrera y año de ingreso..... 46
Consulta 4 · Porcentaje de aprobación por materia..... 46
Consulta 5 · Edad promedio por carrera..... 48
Consulta 6 · Colección auxiliar resumen_carrera..... 48
Consulta 7 · Edad promedio de aprobados por carrera y tipo de institución..... 50
Consulta 8 · Distribución de aprobados por municipio y carrera..... 50
Consulta 9 · Evaluaciones por mes y materia (instituciones públicas)..... 51
Consulta 10 · Top 5 de carreras más demandadas entre 16 y 18 años..... 52

A.2 Consultas 11 a 20 53
Consulta 11 · Historial de desempeño por aspirante 53
Consulta 12 · Distribución por sexo y tipo de institución educativa 54
Consulta 13 · Tasa de aprobación por edad 54
Consulta 14 · Número promedio de intentos por materia 55
Consulta 15 · Historial completo de un aspirante 56
Consulta 16 · Carreras con más aspirantes reprobados en su primer intento 57
Consulta 17 · Top 5 de municipios con más aspirantes 58
Consulta 18 · Tasa de aprobación por tipo de institución educativa 58
Consulta 19 · Carreras más demandadas por departamento 59
Consulta 20 · Evolución de la aprobación por año de ingreso 60
Anexo B. Consultas de Cypher 62
B.1 Consultas G1 a G6 62
Consulta G1 · Carreras más demandadas desde un departamento 62
Consulta G2 · Pares de carreras que comparten aspirantes 63
Consulta G3 · Aspirantes que cambian de carrera de un año al siguiente 64
Consulta G4 · Aprobaron Matemática tras varios intentos, por tipo de institución 65
Consulta G5 · Flujo de aprobados: de qué departamentos a qué carreras 65
Consulta G6 · Aspirantes que postulan varios años y cuántos terminan aprobando 66
B.2 Vistas del explorador de grafos 67
Vista A · Mapa de demanda 67
Vista B · Red de un aspirante 67

1. Introducción y alcance

1.1 Contexto

Cada año la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala (USAC) aplica **pruebas específicas** de Matemática y Computación a los aspirantes de primer ingreso. Los resultados se publican como archivos CSV, un archivo por año de ingreso. En ese formato es difícil hacer consultas complejas, detectar patrones o ver cómo se relacionan los aspirantes, las carreras, las instituciones y los lugares de procedencia.

Este proyecto construye una aplicación que **carga, valida, transforma, almacena, consulta y visualiza** esos datos usando dos modelos de bases de datos NoSQL con funciones distintas:

- **MongoDB** (base de datos documental): guarda las evaluaciones y responde las consultas estadísticas y analíticas (conteos, promedios, porcentajes, rankings).
- **Neo4j** (base de datos de grafos): representa las relaciones entre aspirantes, postulaciones, carreras, materias, instituciones y lugares, y responde consultas de recorridos y patrones.

1.2 Alcance

Qué hace el sistema:

- Carga uno o varios archivos CSV desde la interfaz, los valida y los transforma según reglas documentadas, y reporta cuántos registros se procesaron o rechazaron y por qué.
- Guarda los datos en MongoDB y genera el grafo en Neo4j sin duplicar información si un archivo se carga más de una vez.
- Ofrece 20 consultas analíticas sobre MongoDB y 6 consultas de grafos sobre Neo4j, con gráficas, tablas y el código de cada consulta.
- Permite explorar visualmente el grafo con filtros y buscar aspirantes por varios criterios.
- Maneja los errores de forma controlada: archivos inválidos, bases de datos caídas, consultas con parámetros incorrectos.

Qué no es:

- No es un sistema de admisión: aprobar las pruebas específicas no equivale a ser admitido. El conjunto de datos solo contiene los resultados de estas pruebas.
- No predice resultados ni identifica personas: el identificador del aspirante es anónimo.

1.3 Datos de origen

Los datos son el conjunto **«Pruebas específicas»** del portal de Datos Abiertos de la Facultad de Ingeniería de la USAC, publicado por su Centro de Cálculo e Investigación Educativa (CCIE).

Tabla 2. Características del conjunto de datos

Aspecto	Detalle
Fuente	datosabiertos.ingenieria.usac.edu.gt/dataset/pruebas-especificas
Versión publicada	2.0 (creado el 21/09/2022, actualizado el 22/08/2024). Sin licencia especificada

Aspecto	Detalle
Archivos	10 CSV: `pruebas_especificas_2015.csv` a `pruebas_especificas_2024.csv`
Filas	208,453 (cada fila es una evaluación de un aspirante en una materia)
Codificación	UTF-8 sin BOM, saltos de línea LF, sin filas con número de columnas incorrecto
Aspirantes distintos	40,197 personas, que generan 56,337 postulaciones (aspirante + año de ingreso)

Tabla 3. Campos de cada archivo

Campo	Descripción
`fecha_asignacion`	Fecha en que el aspirante se asignó a la prueba
`sexo`	Sexo registrado (`M`, `F` o `SIN REGISTRO`)
`anio_nacimiento`	Año de nacimiento (puede ser `SIN REGISTRO`)
`materia`	`MATEMATICA`, `COMPUTACION` o `MATEMATICA ITUGS`
`numero_de_fecha_de_evaluacion`	Número de oportunidad dentro del año de ingreso (1 a 4, y 5 en 2023–2024)
`anio_de_ingreso`	Año para el que el aspirante desea ingresar
`aprobacion`	`APROBADO` o `NO APROBADO`
`carrera_objetivo`	Carrera a la que desea ingresar (`NINGUNO` si no se indicó)
`departamento_institucion_educativa`	Departamento de la institución de procedencia
`municipio_institucion_educativa`	Municipio de la institución de procedencia
`tipo_institucion_educativa`	`PRIVADO`, `PUBLICO`, `POR COOPERATIVA`, `MUNICIPAL` o `SIN DATOS`
`correlativo_aspirante`	Identificador anónimo del aspirante

Los datos no están limpios. Los archivos de 2022 a 2024 tienen otra estructura (columnas reordenadas y encabezados cortados) y valores truncados en el identificador, la carrera y el municipio; además hay duplicados y valores especiales (`SIN REGISTRO`, `NINGUNO`, `SIN DATOS`). El capítulo 4 explica cómo se trata cada caso.

1.4 Conceptos clave

Estos términos se usan en todo el documento con el significado siguiente.

Tabla 4. Glosario

Término	Significado
Evaluación	Una fila del CSV: un aspirante, en una materia, en una oportunidad de un año de ingreso.
Oportunidad	Cada una de las fechas del año en que se puede presentar la prueba (4, y 5 en 2023–2024). Un aspirante puede aparecer por primera vez en cualquiera de ellas.
Intento	Cada aparición de un aspirante en una materia. Quien aparece por primera vez en la oportunidad 3 está en su intento 1.

Término	Significado
Postulación	Un aspirante en un año de ingreso. Una persona puede tener varias postulaciones (una por año).
Aspirante / persona	El identificador anónimo (primeros 20 caracteres del correlativo). Puede aparecer en varios años.
Habilitado (aprobado)	Aspirante con Matemática y Computación aprobadas, donde la segunda aprobación se obtuvo dentro de los 2 años de la primera (sección 4.6).
Primer intento	Las evaluaciones de la menor oportunidad en que apareció el aspirante en ese año de ingreso.

2. Arquitectura y tecnologías

2.1 Visión general

La aplicación separa con claridad tres capas: la **interfaz** (frontend), la **lógica de aplicación** (backend o API) y las **bases de datos**. El navegador solo se comunica con la API; nunca se conecta directamente a MongoDB ni a Neo4j.

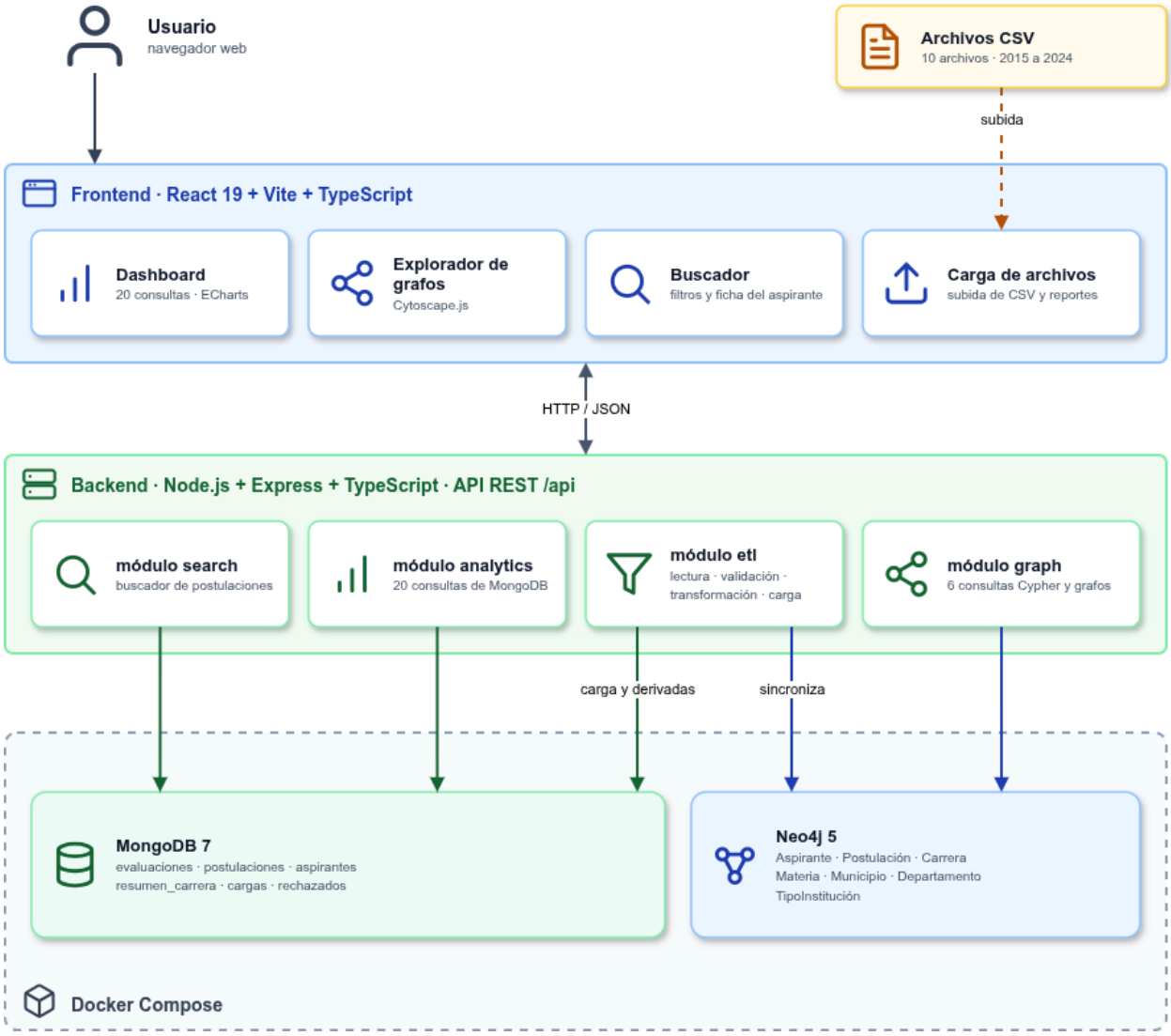


Figura 1. *Arquitectura de la aplicación*

El backend es el único que conoce las bases de datos. Sus responsabilidades, tal como las pide el enunciado, son recibir las solicitudes del frontend, validar los datos recibidos, ejecutar las consultas en MongoDB y en Neo4j, procesar los resultados, enviarlos al frontend y manejar los errores de forma controlada.

El backend se organiza en **cuatro módulos por función**, cada uno con la misma estructura interna (rutas, controlador, servicio):

- **etl**: lectura, validación, transformación y carga de los CSV, reconstrucción de las colecciones derivadas, sincronización de Neo4j y borrado de datos.
- **analytics**: las 20 consultas de MongoDB.
- **graph**: las consultas Cypher y la construcción de los grafos para el explorador.
- **search**: el buscador de postulaciones.

2.2 Tecnologías

Tabla 5. Tecnologías utilizadas y sus versiones

Capa	Tecnología	Versión	Uso
Bases de datos	MongoDB	7 (imagen Docker)	Evaluaciones, derivadas y consultas analíticas
	Neo4j Community	5 (imagen Docker)	Grafo de relaciones y consultas Cypher
Backend	Node.js	20	Entorno de ejecución
	TypeScript / tsx	7.0 / 4.23	Lenguaje con tipos y ejecución directa
	Express	4.22	API REST
	mongodb (driver oficial)	7.7	Acceso a MongoDB sin ORM
	neo4j-driver	6.2	Acceso a Neo4j con Cypher
	csv-parse / multer	7.0 / 2.5	Lectura de CSV en streaming y subida de archivos
Frontend	React / React Router	19.3 / 7.18	Interfaz y navegación
	Vite	8.3	Servidor de desarrollo y empaquetado
	Apache ECharts	6.1	Gráficas del dashboard
	Cytoscape.js	3.34	Visualización de grafos
Infraestructura	Docker Compose	—	Levanta MongoDB y Neo4j con volúmenes persistentes

Por qué estas tecnologías

Tabla 6. Alternativas consideradas

Decisión	Se eligió	Alternativas	Razón principal
Backend	Node.js + TypeScript + Express	Python + FastAPI; Java + Spring Boot; .NET	Los pipelines de MongoDB son objetos de JavaScript casi idénticos a su sintaxis; los drivers oficiales son maduros; permite leer CSV en streaming; un solo lenguaje con el frontend. Spring Boot y .NET añaden mucho código base y ocultan las consultas detrás de abstracciones.
Acceso a datos	Drivers oficiales, sin Mongoose ni OGM	Mongoose; Spring Data	Las consultas deben verse y poder modificarse en la defensa; un ORM las esconde.
Frontend	React + Vite	Vue; Angular; HTML puro	Mayor ecosistema de gráficas y grafos; arranque rápido. Angular es pesado para este alcance.

Decisión	Se eligió	Alternativas	Razón principal
Gráficas	Apache ECharts	Recharts; Chart.js; Plotly	Una sola librería cubre barras, pastel, líneas, apiladas y eje secundario, que es lo que piden las 20 consultas.
Grafos	Cytoscape.js	vis-network; react-force-graph; D3	Buenos diseños automáticos, estilos por tipo de nodo y selección con resaltado de vecinos.
Despliegue de bases	Docker Compose	Instalación manual	Reproducible: con un comando cualquiera levanta las dos bases con la misma versión.

2.3 Estructura del repositorio

La estructura de carpetas refleja la arquitectura: cada módulo del backend y cada sección del frontend tiene su propio directorio.

```
proyecto2/
├── docker-compose.yml        MongoDB y Neo4j
├── .env.example              plantilla de variables de entorno
├── data/                     los 10 CSV (no se versionan)
├── docs/                     manuales, modelos, reglas de ETL, consultas
│   └── generador/            scripts que producen este manual (.docx)
├── backend/
│   ├── scripts/              carga por línea de comandos, reconstrucción, borrado,
│   │                         generación de documentación
│   ├── tests/                33 pruebas unitarias
│   └── src/
│       ├── config/           conexiones a MongoDB y Neo4j, variables de entorno
│       ├── db/               índices de MongoDB y restricciones de Neo4j
│       ├── shared/           errores, middlewares (errores, subida de archivos), logger
│       └── modules/
│           ├── etl/          pipeline/ (lector, validador, transformador, deduplicación,
│           │                 aprobación, cargadores), servicios de derivación y borrado
│           ├── analytics/    queries/ (q01 … q20, una consulta por archivo)
│           ├── graph/        cypher/ (g01 … g06) y construcción de grafos
│           └── search/       buscador
└── frontend/
    └── src/
        ├── api/              cliente HTTP y una función por endpoint
        ├── components/       gráficas (ECharts), ejecutor genérico de consultas, tablas
        ├── features/         dashboard, graph-explorer, search, upload
        ├── hooks/ types/ utils/
        └── app/              menú, rutas y aviso de conexión
```

2.4 Principios de diseño

- **Dependencias en una sola dirección:** ruta → controlador → servicio → base de datos. Un controlador nunca habla con las bases de datos.
- **Una consulta por archivo.** Cada una de las 20 consultas de MongoDB y de las 6 de Cypher vive en su propio archivo, con su descripción, parámetros, código y tipo de gráfica. Esto facilita explicarla y modificarla.
- **La interfaz se guía por metadatos.** Cada consulta declara cómo debe graficarse (tipo de gráfica y ejes), de modo que una sola pantalla genérica muestra las 26 consultas.
- **Lo difícil se calcula una vez, en el ETL.** Reglas como «habilitado con vigencia de 2 años por materia» comparan fechas de años distintos; se calculan al cargar y se guardan como campos, de modo que las consultas quedan cortas.
- **Todo es repetible (idempotente).** Cargar el mismo archivo dos veces, o sincronizar Neo4j varias veces, no duplica datos.

- **Los errores se controlan.** Toda respuesta de error tiene el mismo formato, y el servidor sigue vivo si una base de datos se cae.

2.5 Decisiones técnicas relevantes

La tabla resume las decisiones de diseño más importantes, con la alternativa que se descartó y la razón. Los capítulos 4, 5 y 6 las desarrollan.

Tabla 7. Registro de decisiones técnicas

Decisión	Alternativa descartada	Razón
Identificar al aspirante con los **primeros 20 caracteres** del correlativo	Usar el correlativo completo	En 2022–2024 el identificador viene truncado (20 a 23 caracteres). Los 20 primeros no tienen colisiones en 2015–2021 y todos los de 2022+ coinciden con el prefijo de uno anterior.
Una **colección** `evaluaciones` (una fila por evaluación) más colecciones **derivadas**	Una sola colección con aspirantes y arreglos anidados	Las consultas por evaluación se mantienen simples; las derivadas guardan el estado calculado.
`_id` de `evaluaciones` **= clave natural** (año, aspirante, oportunidad, materia)	`ObjectId` más un índice único	Hace que recargar un archivo sea idempotente sin trabajo extra: un duplicado produce un error de clave repetida que se cuenta como «omitido».
«Postulación» como nodo en Neo4j	Colgar carrera, edad y estado directamente del aspirante	Esos datos cambian de un año a otro; con el nodo intermedio el año es una propiedad y se puede recorrer la historia de una persona.
Cada **evaluación es una relación** con propiedades	Un nodo por evaluación	Habría 207,606 nodos sin nada más colgando de ellos; la relación con `oportunidad`, `aprobada` e `intento` es suficiente.
Vigencia de 2 años por materia para el estado de aprobación	Contar solo lo aprobado en el mismo año, o acumular sin límite	Lo respalda el artículo 43 del Reglamento del SUN y una constancia de resultados del SUN; acumular sin límite no tiene respaldo normativo.
Corregir `QUEZALTENANGO` a `QUETZALTENANGO` en el ETL	Filtrar por la grafía errónea en las consultas	Es el nombre oficial y el propio dataset lo escribe así en el municipio; corregirlo en el origen evita nodos y grupos duplicados.
Completar municipios truncados con un catálogo	Dejarlos truncados	Evita nodos duplicados en Neo4j (por ejemplo `SAN LUCAS SACATEPEQ` junto a `SAN LUCAS SACATEPEQUEZ`).
Bloqueo de operaciones pesadas y confirmación para borrar	Permitir operaciones simultáneas	Cargar y borrar a la vez dejaría las bases en estados mezclados.

3. Instalación y configuración

3.1 Requisitos

- Node.js 20 o superior y npm.
- Docker y Docker Compose (para MongoDB y Neo4j).
- Los 10 archivos `pruebas_especificas_AAAA.csv` (2015 a 2024) copiados en la carpeta `data/`. Se descargan del portal de Datos Abiertos de la Facultad de Ingeniería; no se versionan en el repositorio.

3.2 Puesta en marcha

Desde la raíz del proyecto:

1. **Variables de entorno.** Copiar la plantilla y poner una contraseña propia para Neo4j (mínimo 8 caracteres).

   ```
   cp .env.example .env
   # editar .env y cambiar NEO4J_PASSWORD
   ```

2. **Bases de datos.** Levantar MongoDB y Neo4j con Docker Compose.

   ```
   docker compose up -d mongo neo4j
   ```

3. **Backend.** Instalar dependencias.

   ```
   cd backend
   npm install
   ```

4. **Carga de datos.** Procesar los 10 CSV: MongoDB, colecciones derivadas y Neo4j (tarda cerca de un minuto). También puede hacerse desde la interfaz.

   ```
   npm run load
   ```

5. **API.** Iniciar el servidor en http://localhost:3000.

   ```
   npm start
   ```

6. **Interfaz.** En otra terminal, instalar e iniciar el frontend en http://localhost:5173.

   ```
   cd frontend
   npm install
   npm run dev
   ```

3.3 Variables de entorno

Todas se definen en el archivo `.env` de la raíz (ignorado por Git). La plantilla `.env.example` trae valores por defecto, salvo la contraseña de Neo4j, que es obligatoria.

Tabla 8. Variables de entorno

Variable	Valor por defecto	Descripción
`BACKEND_PORT`	3000	Puerto de la API
`MONGO_PORT`	27017	Puerto de MongoDB expuesto por Docker
`MONGO_URI`	mongodb://localhost:27017	Cadena de conexión que usa el backend
`MONGO_DB`	admision_usac	Nombre de la base de datos

Variable	Valor por defecto	Descripción
`NEO4J_HTTP_PORT` / `NEO4J_BOLT_PORT`	7474 / 7687	Puertos del navegador de Neo4j y del protocolo Bolt
`NEO4J_URI`	bolt://localhost:7687	Conexión del backend a Neo4j
`NEO4J_USER`	neo4j	Usuario de Neo4j
`NEO4J_PASSWORD`	(obligatoria)	Contraseña de Neo4j. Docker la usa al crear la base por primera vez

3.4 Servicios y puertos

Tabla 9. Servicios de la aplicación

Servicio	Dirección	Notas
Interfaz web	http://localhost:5173	Vite en modo desarrollo
API	http://localhost:3000/api	`GET /api/health` informa el estado de las dos bases de datos
MongoDB	mongodb://localhost:27017	Sin usuario ni contraseña (uso local). Se puede explorar con MongoDB Compass
Neo4j Browser	http://localhost:7474	Usuario `neo4j` y la contraseña del `.env`
Neo4j (Bolt)	bolt://localhost:7687	Conexión del backend

Los datos de ambas bases se guardan en volúmenes de Docker (`mongo-data` y `neo4j-data`), por lo que sobreviven a reiniciar los contenedores.

3.5 Comandos disponibles

Tabla 10. Comandos del backend (carpeta backend/)

Comando	Qué hace
`npm start`	Inicia la API (crea los índices y las restricciones al arrancar)
`npm run load -- [archivos]`	Carga por línea de comandos los CSV indicados; sin argumentos, todos los de `data/`
`npm run reconstruir`	Recalcula las colecciones derivadas y sincroniza Neo4j sin leer los CSV
`npm run reiniciar -- --si`	**Borra** los datos de MongoDB y Neo4j (sin `--si` no hace nada)
`npm run catalogo`	Regenera el catálogo de municipios usado para completar nombres truncados
`npm run consultas` / `npm run cypher`	Ejecutan las 20 consultas de MongoDB / las 6 de Cypher y muestran filas y tiempos
`npm run doc:consultas` / `npm run doc:cypher`	Regeneran la documentación de las consultas con su código y resultados reales
`npm test`	Ejecuta las 33 pruebas unitarias
`npm run build`	Revisa los tipos con el compilador de TypeScript

Tabla 11. Comandos del frontend (carpeta frontend/)

Comando	Qué hace
`npm run dev`	Inicia el servidor de desarrollo en el puerto 5173
`npm run build`	Revisa los tipos y genera la versión de producción en `dist/`

3.6 Solución de problemas

Tabla 12. Problemas frecuentes

Síntoma	Causa probable	Solución
El backend no inicia y dice `EADDRINUSE`	Ya hay otro backend usando el puerto 3000	Detener el proceso anterior o iniciar con otro puerto: `BACKEND_PORT=3001 npm start`
Error `Falta la variable de entorno NEO4J_PASSWORD`	No existe el `.env`	Copiar `.env.example` a `.env` y completar la contraseña
Neo4j responde «authentication failure»	La contraseña del `.env` cambió después de crear el volumen de Neo4j	Volver a la contraseña original, o recrear el volumen con `docker compose down -v` (borra los datos)
La interfaz muestra «No hay conexión con el servidor»	El backend no está en ejecución	Iniciarlo con `npm start` en `backend/`
Aviso rojo «Neo4j no está disponible»	El contenedor de Neo4j está detenido	`docker compose up -d neo4j`; la interfaz se recupera sola
El dashboard dice «No hay datos cargados»	Todavía no se han cargado los CSV, o se borraron	Ir a «Carga de archivos» y subirlos, o ejecutar `npm run load`
`npm run load` no encuentra archivos	Los CSV no están en `data/`	Copiar los 10 archivos a la carpeta `data/` de la raíz

4. Proceso ETL

ETL significa *extraer, transformar y cargar*. Este proceso toma los CSV, los limpia según reglas explícitas y los guarda en MongoDB y en Neo4j. Se implementó en el módulo `etl` del backend (`backend/src/modules/etl/`), y se ejecuta igual desde la interfaz web o por línea de comandos (`npm run load`).

4.1 Visión general del flujo

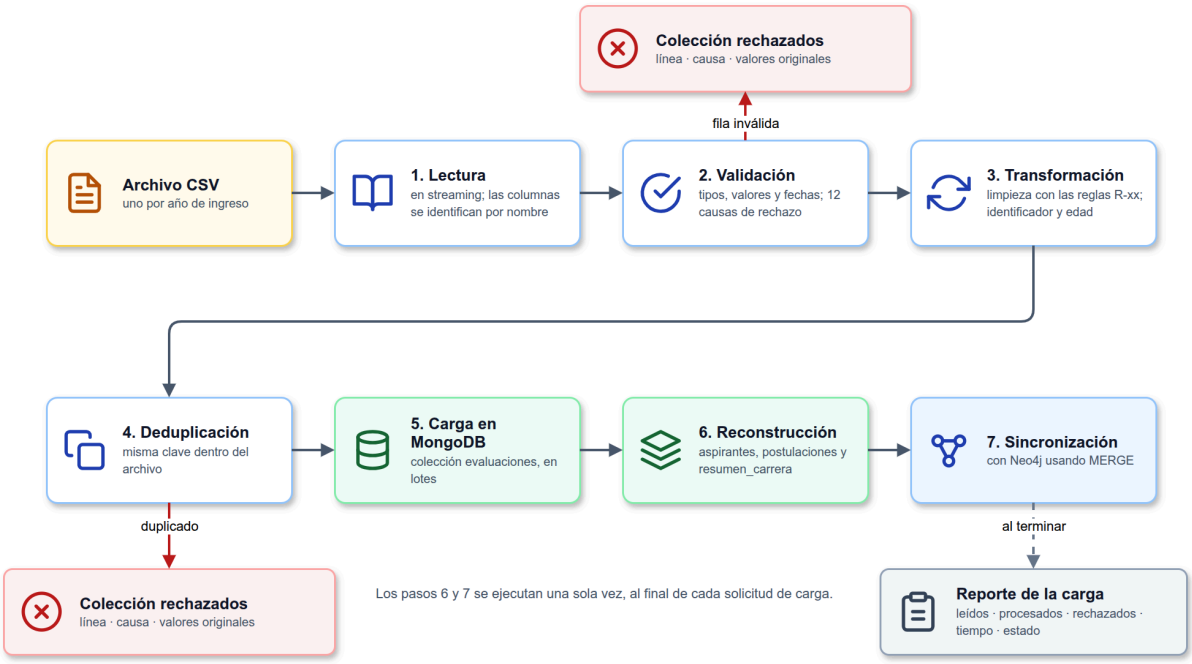


Figura 2. *Flujo del proceso ETL*

1. **Lectura.** Cada archivo se lee en *streaming* (fila por fila, sin cargarlo completo en memoria). Las columnas se identifican por su nombre, no por su posición.
2. **Validación.** Cada fila se revisa: tipos, valores permitidos, fechas reales. Una fila inválida se rechaza con su causa.
3. **Transformación.** Se normalizan valores (carrera, municipio, departamento), se calcula la edad y se descompone el identificador.
4. **Deduplicación.** Dentro del archivo, dos filas con la misma clave se consideran duplicadas y solo se conserva una.
5. **Carga en MongoDB.** Las filas válidas se insertan en la colección `evaluaciones` por lotes.
6. **Reconstrucción de derivadas.** Con todas las evaluaciones a la vista se calculan los estados de aprobación y se generan `aspirantes`, `postulaciones` y `resumen_carrera`.
7. **Sincronización de Neo4j.** El grafo se genera a partir de esas colecciones derivadas, de modo que ambas bases comparten exactamente los mismos estados.

Los pasos 6 y 7 se ejecutan **una sola vez al final** de cada solicitud de carga, porque el estado de un aspirante depende de todos sus años: cargar el archivo de 2020 puede cambiar el estado de un aspirante de 2019.

4.2 Estructura de los archivos de origen

Los archivos de 2015 a 2021 y los de 2022 a 2024 no tienen la misma estructura. Si el programa leyera las columnas por posición, interpretaría mal los datos de 2022 en adelante.

Tabla 13. Diferencias entre los archivos

Aspecto	2015–2021	2022–2024
Posición de `correlativo_aspirante`	Última columna	Segunda columna
Encabezados	Completos	Algunos cortados: `numero_de_fecha_de_evaluaci`, `departamento_institucion_ed`, `municipio_institucion_`
Identificador del aspirante	32 caracteres hexadecimales	Truncado a 20–23 caracteres
Carrera	Nombre completo	Truncada (`INGENIERIA EN CIENCIAS Y` a 25 caracteres; Mecánica Industrial y Eléctrica quedan como `INGENIERIA MECANICA`)
Municipio	Completo (la capital incluye la zona)	Truncado a 19 caracteres (la capital pierde la zona)

Tabla 14. Reglas de lectura

Código	Regla
R-E1	Las columnas se leen **por nombre**, nunca por posición.
R-E2	Un encabezado es válido si es prefijo (sin distinguir mayúsculas) de **exactamente un** nombre esperado; así se aceptan los encabezados cortados.
R-E3	Si falta una columna o un encabezado no se reconoce, el archivo termina como `FALLIDA` y no se carga ninguna fila.
R-E4	Se usa un lector CSV estándar: hay valores entrecomillados y otros no. **No se recortan espacios** al leer, porque en algunos valores el espacio final tiene significado (sección 4.4).
R-E5	Una fila con distinto número de columnas que el encabezado se rechaza (`ESTRUCTURA_INVALIDA`).

4.3 Validación y causas de rechazo

El backend distingue dos resultados para una fila: el **rechazo** (la fila no se carga y se guarda con su causa) y la **advertencia** (la fila se carga pero queda marcada). Nada se elimina ni se corrige sin dejar registro.

Tabla 15. Causas de rechazo

Causa	Cuándo se produce
`ESTRUCTURA_INVALIDA`	La fila tiene distinto número de columnas que el encabezado
`CAMPO_REQUERIDO_VACIO`	Alguno de los 12 campos viene vacío

Causa	Cuándo se produce
FECHA_INVALIDA	fecha_asignacion no es una fecha real (por ejemplo 2021-02-30)
SEXO_INVALIDO	El sexo no es M, F ni SIN REGISTRO
ANIO_NACIMIENTO_INVALIDO	No es un año de 4 dígitos ni SIN REGISTRO
MATERIA_INVALIDA	La materia no es una de las tres conocidas
OPORTUNIDAD_INVALIDA	El número de oportunidad no es un entero positivo
ANIO_INGRESO_INVALIDO	anio_de_ingreso no es un año de 4 dígitos
APROBACION_INVALIDA	El valor no es APROBADO ni NO APROBADO
TIPO_INSTITUCION_INVALIDO	El tipo de institución no es uno de los conocidos
CORRELATIVO_INVALIDO	El correlativo no tiene entre 20 y 32 caracteres hexadecimales
DUPLICADO	Otra fila del mismo archivo tiene la misma clave (sección 4.5)

En los 10 archivos reales la única causa de rechazo que se presenta es DUPLICADO (847 filas). Las otras 11 causas se comprobaron con un archivo de prueba con una fila inválida de cada tipo (capítulo 9).

4.4 Transformación

4.4.1 Identificador del aspirante

Tabla 16. Reglas del identificador

Código	Regla
R-ID1	aspirante_id = los **primeros 20 caracteres** del correlativo, en minúscula. El valor original se conserva en correlativo_original.
R-ID2	El correlativo debe cumplir ^[0-9a-f]{20,32}$; si no, se rechaza.
R-ID3	Si un mismo aspirante muestra sexos distintos entre años, se usa el del año más reciente y se marca CONFLICTO_SEXO.

Por qué 20 caracteres. Se verificó que es seguro:

- Los 29,418 identificadores completos de 2015–2021 producen 29,418 prefijos de 20 caracteres distintos: no hay colisiones.
- Los 2,174 identificadores de 2022–2024 que coinciden con uno anterior son, sin excepción, prefijo exacto del de 32 caracteres.
- No hay conflictos de año de nacimiento entre los 11,096 aspirantes que reaparecen con ese dato conocido. Solo 2 de 40,143 aspirantes cambian de sexo entre años.
- La tasa de reaparición entre años consecutivos no se rompe en el cambio de formato (27.4 % de 2020 a 2021 y 24.4 % de 2021 a 2022).

Límite: con identificadores anónimos no es posible detectar a una misma persona con dos códigos distintos, ni confirmar que un código no se haya reutilizado.

4.4.2 Carrera

Tabla 17. Reglas de la carrera objetivo

Código	Regla	Casos
R-C1	`NINGUNO` pasa a carrera nula (`SIN_CARRERA`).	20,275 de las 20,277 filas de 2015
R-C2	`INGENIERIA EN CIENCIAS Y` (truncada) se completa a `INGENIERIA EN CIENCIAS Y SISTEMAS`.	24,221 (2022–2024)
R-C3	`INGENIERIA MECANICA` (**con espacio final**) pasa a `INGENIERIA MECANICA (INDUSTRIAL O ELECTRICA, SIN PRECISAR)`, porque no se puede saber cuál de las dos era.	3,789 (2022–2024)
R-C4	R-C2 y R-C3 se aplican al valor **crudo, antes de recortar espacios**. Si se recortara primero, `INGENIERIA MECANICA` se confundiría con `INGENIERIA MECANICA`, que es una carrera distinta.	—
R-C6	`ITUGS` se conserva, pero no participa en el estado de aprobación.	22 filas
R-C8	La carrera es atributo de la **evaluación**, no del aspirante: 5,616 postulaciones tienen más de una. En agrupaciones por carrera, una postulación cuenta una vez por cada carrera distinta.	5,616

4.4.3 Geografía e institución

Tabla 18. Reglas de ubicación e institución

Código	Regla	Casos
R-G1	`SIN DATOS` en tipo, departamento y municipio (siempre aparecen juntos) pasa a nulo.	4,005
R-G4	`GUATEMALA - CIUDAD CAPITAL ZONA N` se separa en municipio `GUATEMALA` y zona `N`. En 2022–2024 la zona se pierde por el truncado y queda nula (`ZONA_NO_DISPONIBLE`).	25,579 sin zona
R-G5	**Municipios truncados** (19 caracteres o con espacio final): se completan buscando en un catálogo de 2015–2021, dentro del mismo departamento, un único nombre que empiece igual. Si no hay candidato único se conserva recortado y se marca.	2,439 recuperados; 23 sin candidato
R-G6	La clave del municipio es **(departamento, municipio)**: hay nombres repetidos en departamentos distintos (`SAN JOSE`, `SANTA LUCIA`).	—
R-G8	**Departamento mal escrito:** el dataset escribe `QUEZALTENANGO`; el nombre oficial es `QUETZALTENANGO`. Se corrige y se marca `DEPARTAMENTO_CORREGIDO`. Se compararon los 22 departamentos oficiales y es el único que difiere.	2,176 filas

4.4.4 Edad

- **R-ED1:** `edad = anio_de_ingreso − anio_nacimiento`. Así es la misma para todos los intentos de un ciclo; usar el año de la fecha de la prueba daría edades distintas entre intentos.
- **R-ED2:** una edad es válida entre 13 y 70 años; fuera de ese rango queda nula (`EDAD_FUERA_DE_RANGO`) y la fila se conserva. En los datos solo 4 filas quedan fuera (un aspirante con año de nacimiento 1900).
- **R-ED3:** las postulaciones sin edad se excluyen de las consultas que usan la edad.

4.5 Duplicados y anomalías

La **clave de una evaluación** es el conjunto (año de ingreso, aspirante, oportunidad, materia): no puede haber dos evaluaciones iguales de la misma persona. Se detectaron 847 filas sobrantes en 818 grupos repetidos, concentradas en 2016–2020.

Tabla 19. Tratamiento de duplicados y anomalías

Caso	Cantidad	Tratamiento
Fila repetida (misma clave)	818 grupos, 847 filas sobrantes (805 idénticos; 8 difieren en la fecha; 5 en fecha y carrera)	**R-D1:** se conserva la de fecha más temprana. Las demás se rechazan con causa `DUPLICADO`, guardando línea, archivo y los campos que difieren. Ninguna tiene resultado contradictorio.
Recarga de un archivo	—	**R-D2:** las filas cuya clave ya existe no se reinsertan y se reportan como «omitidas», no como rechazadas.
Reprobación posterior a una aprobación de la misma materia	184 aspirante-materia (0.6 %)	**R-D3:** la evaluación se conserva, la materia sigue aprobada y se marca. **Una aprobación nunca se revierte.**
Segunda aprobación de la misma materia	18 (todas en el mismo año)	**R-D4:** se usa la primera y se marca.
Misma materia en varios años	17,406 de 78,723 aspirante-materia	No es anomalía: cada año de ingreso se evalúa por separado.

4.6 Aprobación y primer intento

Decidir qué significa «aprobado» fue la decisión conceptual más importante del proyecto, porque un aspirante puede presentar cada materia varias veces, en distintas oportunidades e incluso en distintos años. Se definieron **tres niveles**.

Tabla 20. Los tres niveles de aprobación

Nivel	Qué es	Regla	Dónde se guarda	Cifra real
1. Evaluación	Una fila del CSV	Es el resultado tal cual (`APROBADO` o no)	`evaluaciones.aprobada`	207,606 evaluaciones (29,697 aprobadas)
2. Materia del aspirante	Aspirante + año + materia	Aprobada si **algún** intento fue aprobado	`postulaciones.materias[]`	55,865 en Matemática y 45,476 en Computación
3. Estado del aspirante	Aspirante + año (postulación)	Aprobó las dos materias con vigencia de 2 años por materia	`postulaciones.estado` y `aprobado`	8,471 aprobados

Un ejemplo de los datos reales: un aspirante reprueba Computación y Matemática en la oportunidad 1, aprueba Computación en la 2, reprueba Matemática en la 2 y la 3, y la aprueba en la 4. A nivel de evaluación hay seis filas (dos aprobadas); a nivel de materia, ambas están aprobadas (Computación al 2.º intento, Matemática al 4.º); y su estado final es **aprobado**. Este caso está cubierto por una prueba automática.

4.6.1 Vigencia de las aprobaciones

Muchos aspirantes presentan en un año solo una de las dos materias, porque ya habían aprobado la otra en un año anterior (73.8 % de esos casos). Por eso el estado final no puede calcularse con las filas de un solo año. La pregunta es cuánto tiempo vale una aprobación.

- **Norma:** el artículo 43 del Reglamento del Sistema de Ubicación y Nivelación (SUN), vigente desde el 5 de junio de 2009, establece que el resultado satisfactorio de las pruebas específicas tiene vigencia de **dos años**.
- **Práctica:** una constancia de resultados del SUN aportada por el autor lista cada materia con su propia fecha y su propio vencimiento, exactamente a dos años de la fecha de la prueba. Eso respalda que la vigencia es **por materia** y se cuenta desde la fecha de la prueba. La constancia no se incluye por contener datos personales, y es del Centro Universitario de Occidente, no de la Facultad de Ingeniería central, aunque ambas se rigen por el mismo reglamento.

Regla adoptada: una aprobación de una materia vence dos años calendario después de la fecha de la prueba (`fecha_asignacion`). Un aspirante está **habilitado** cuando tiene una aprobación de Matemática y una de Computación tales que la más reciente se obtuvo dentro de los dos años de la más antigua. Si hay varios pares válidos se toma la habilitación más temprana, y la persona se cuenta como aprobada una sola vez, en el año de su habilitación.

4.6.2 Indicador principal y complementario

Se guardan dos indicadores, para que se vea cuánto depende el resultado de la regla elegida:

- **(c) Principal: habilitado con vigencia por materia.** Es el valor por defecto de las consultas de «aspirantes aprobados» (3, 7, 8, 13 y 18).
- **(a) Complementario: ambas materias aprobadas dentro del mismo año.** Es la cota que se puede verificar usando solo el CSV, sin supuestos normativos.

Tabla 21. Sensibilidad del resultado a la regla de vigencia (total de los 10 años)

Criterio	Aprobados	% de las postulaciones
(a) Solo lo aprobado dentro del propio año	7,011	12.4 %
Acumulado, vigencia de 1 año	7,884	14.0 %
(c) Acumulado, vigencia de 2 años por materia (elegido)	**8,471**	**15.0 %**
Acumulado, vigencia de 3 años	8,570	15.2 %
Acumulado, sin límite de años (descartado)	8,659	15.4 %

Elegir 730 días o ningún límite cambia el resultado solo en 189 casos (2.2 %): la conclusión es robusta frente a esa decisión. En cambio, usar únicamente el año dejaría de contar a 1,460 aspirantes que completaron entre dos ciclos.

Tabla 22. Indicadores por año de ingreso

Año	Postulaciones	(c) Habilitados	%	(a) Mismo año	%
2015	6,148	910	14.8 %	910	14.8 %
2016	6,428	908	14.1 %	628	9.8 %

Año	Postulaciones	(c) Habilitados	%	(a) Mismo año	%
2017	5,976	1,012	16.9 %	839	14.0 %
2018	5,738	1,071	18.7 %	896	15.6 %
2019	5,310	1,216	22.9 %	857	16.1 %
2020	4,997	986	19.7 %	774	15.5 %
2021	6,861	449	6.5 %	354	5.2 %
2022	6,380	447	7.0 %	386	6.1 %
2023	4,638	765	16.5 %	713	15.4 %
2024	3,861	707	18.3 %	654	16.9 %
Total	**56,337**	**8,471**	**15.0 %**	**7,011**	**12.4 %**

4.6.3 Estado de cada postulación

Tabla 23. Estados posibles y su cantidad

Estado	Condición	Postulaciones
`APROBADO`	Se habilitó en el año de la postulación	8,471
`YA_HABILITADO`	Se habilitó en un año anterior y reaparece	2
`PARCIAL`	No habilitado, pero con al menos una materia aprobada hasta ese año (informativo)	20,928
`NO_APROBADO`	Ninguna materia aprobada hasta ese año	26,936

4.6.4 Primer intento

El primer intento de un aspirante **no es necesariamente la oportunidad 1**: puede aparecer por primera vez en cualquier oportunidad (el 40 % de las postulaciones empieza en la 2, 3, 4 o 5). Se define como las evaluaciones de la **menor oportunidad** que tuvo en ese año de ingreso, y se considera «reprobado» si falló **al menos una materia** en ese primer intento. Existe una variante estricta (reprobar todas), disponible como parámetro de la consulta 16.

4.7 Carga en las bases de datos

4.7.1 MongoDB

- Las filas válidas se insertan en lotes de 5,000 con `insertMany` y `ordered: false`: un error en una fila no detiene el resto del lote.
- El `_id` de cada evaluación es su clave natural. Si ya existe, MongoDB responde con un error de clave repetida (código 11000); el cargador lo cuenta como fila «omitida» en lugar de fallar.
- Al terminar todos los archivos, el servicio de derivación lee las evaluaciones ordenadas por aspirante (usando un índice), calcula en memoria el estado de cada persona y escribe las colecciones `aspirantes` y `postulaciones` en lotes de 1,000 con `replaceOne` y `upsert`. Por último ejecuta la consulta 6 para generar `resumen_carrera`.

4.7.2 Neo4j

Neo4j se llena **después** de MongoDB, a partir de las colecciones derivadas. Se usan consultas con `UNWIND` y `MERGE` sobre claves únicas, en lotes de 2,000 postulaciones y 5,000 evaluaciones, lo que evita crear nodos duplicados y permite repetir la sincronización sin efectos secundarios. Los enteros se envían como `Integer` de Neo4j, porque por defecto el driver envía los números de JavaScript como decimales. El modelo completo está en el capítulo 6.

4.8 Reporte de la carga y resultados

Para cada archivo, la interfaz y la API informan el nombre, la cantidad de registros leídos, los procesados correctamente, los rechazados (con su causa y línea), los omitidos por estar ya cargados, el tiempo y el estado final (`COMPLETADA`, `COMPLETADA_CON_ERRORES` o `FALLIDA`). Se cumple siempre que leídos = procesados + rechazados + omitidos.

Tabla 24. Resultado real de cargar los 10 archivos

Año	Leídos	Procesados	Rechazados	Omitidos	Tiempo	Estado
2015	20,277	20,266	11	0	1.1 s	Con errores
2016	22,137	21,987	150	0	1.1 s	Con errores
2017	21,581	21,398	183	0	1.0 s	Con errores
2018	20,871	20,678	193	0	960 ms	Con errores
2019	17,865	17,835	30	0	822 ms	Con errores
2020	17,931	17,665	266	0	817 ms	Con errores
2021	30,644	30,642	2	0	1.4 s	Con errores
2022	24,237	24,231	6	0	1.1 s	Con errores
2023	19,047	19,047	0	0	803 ms	Completada
2024	13,863	13,857	6	0	609 ms	Con errores
Total	**208,453**	**207,606**	**847**	**0**	**9.8 s**	

Esta tabla corresponde únicamente a los archivos en MongoDB; la sincronización de Neo4j toma aproximadamente 36 segundos para los 10 archivos en total y se informa aparte. Todos los rechazos son duplicados.

Tabla 25. Advertencias generadas (evaluaciones cargadas)

Advertencia	Significado	Evaluaciones
`ZONA_NO_DISPONIBLE`	La zona de la capital se perdió por truncado (2022–2024)	25,575
`CARRERA_COMPLETADA`	Carrera truncada completada (Ciencias y Sistemas)	24,214
`SIN_CARRERA`	La carrera venía como `NINGUNO`	20,264
`SIN_ANIO_NACIMIENTO`	Año de nacimiento `SIN REGISTRO`	19,393
`INSTITUCION_SIN_DATOS`	Institución `SIN DATOS`	3,990
`CARRERA_AMBIGUA`	`INGENIERIA MECANICA` truncada: puede ser Industrial o Eléctrica	3,789
`MUNICIPIO_RECUPERADO`	Municipio truncado completado con el catálogo	2,437

Advertencia	Significado	Evaluaciones
`DEPARTAMENTO_CORREGIDO`	`QUEZALTENANGO` corregido a `QUETZALTENANGO`	2,163
`SEXO_SIN_REGISTRO`	Sexo `SIN REGISTRO`	185
`MUNICIPIO_TRUNCADO`	Municipio truncado sin candidato en el catálogo	23
`EDAD_FUERA_DE_RANGO`	Edad fuera de 13–70 años	4

4.9 Recarga, reconstrucción y borrado

- **Recargar un archivo** no duplica nada: todas sus filas válidas se reportan como omitidas y no se reconstruyen las derivadas.
- **Reconstruir** (`npm run reconstruir` o `POST /api/etl/reconstruir`) recalcula las colecciones derivadas y sincroniza Neo4j sin leer los CSV. Sirve si una sincronización anterior falló.
- **Borrar todos los datos** (botón de la interfaz, `DELETE /api/etl/datos?confirmar=si` o `npm run reiniciar -- --si`) elimina las seis colecciones y todo el grafo para volver a cargar desde cero. Se protege con una confirmación explícita, se niega a actuar si alguna de las bases no responde y no puede ejecutarse mientras haya una carga en curso. Los CSV originales no se tocan.

5. Modelo de MongoDB

MongoDB guarda los registros y responde las 20 consultas analíticas. En una base documental los datos se guardan como **documentos** (JSON) agrupados en **colecciones**. Una decisión central es cuándo *embeber* datos relacionados dentro del documento (un arreglo de subdocumentos) y cuándo *referenciarlos* en una colección aparte.

5.1 Colecciones

El diseño parte de las tres «unidades» que piden las consultas: la **evaluación** (una fila del CSV), la **postulación** (un aspirante en un año) y la **persona** (un aspirante en todos sus años). Cada unidad tiene su colección.

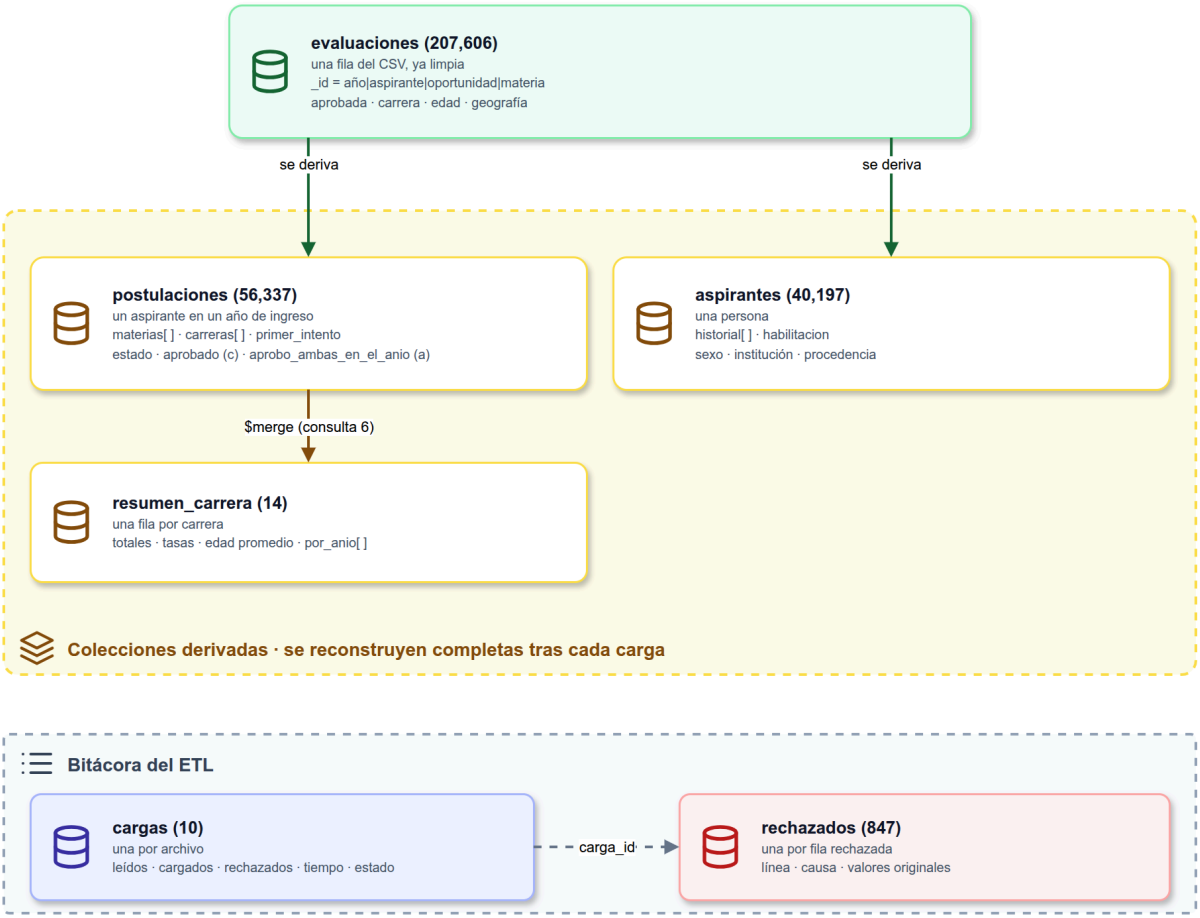


Figura 3. *Colecciones de MongoDB y cómo se relacionan*

Tabla 26. Colecciones de MongoDB

Colección	Documentos	Qué es	Origen
`evaluaciones`	207,606	Una evaluación ya limpia (el «hecho» base)	Se carga desde el CSV
`postulaciones`	56,337	Un aspirante en un año, con sus materias, primer intento y estado de aprobación	Derivada de `evaluaciones`

Colección	Documentos	Qué es	Origen
`aspirantes`	40,197	Una persona con su historial completo	Derivada de `evaluaciones`
`resumen_carrera`	14	Resumen por carrera (consulta 6)	Derivada de `postulaciones`
`cargas`	10	Bitácora de cada archivo procesado	Generada por el ETL
`rechazados`	847	Filas rechazadas, con línea, causa y valores originales	Generada por el ETL

Por qué derivar colecciones. El estado de aprobación compara fechas de aprobaciones de años distintos. Calcularlo una vez en el ETL, con todo el historial a la vista, y guardarlo como un campo hace que las consultas sean pipelines cortos. Las colecciones derivadas se reconstruyen completas después de cada carga, porque un archivo nuevo puede cambiar el estado de aspirantes de años anteriores.

Embeber o referenciar

Tabla 27. Datos embebidos

Dato embebido	Dónde	Tamaño máximo medido
Materias de una postulación	`postulaciones.materias`	3 elementos
Carreras de una postulación	`postulaciones.carreras`	4 (5,616 postulaciones tienen más de una; 6,146 ninguna)
Historial de una persona	`aspirantes.historial`	38 evaluaciones (el 99 % tiene 16 o menos)

Se embebe cuando los datos se leen siempre juntos y su tamaño es pequeño y acotado. Las evaluaciones, en cambio, **no** se embeben dentro de las postulaciones: son la colección grande, se consultan por sí solas (consultas 2, 4, 9 y 20) y duplicarlas en cada lectura sería costoso. Solo `aspirantes.historial` guarda una copia compacta, para leer un historial completo con una sola consulta.

5.2 Colección evaluaciones

Una evaluación de un aspirante, en una materia, en una oportunidad. Su `_id` es la **clave natural** `año|aspirante|oportunidad|materia`, lo que hace idempotente la carga. El ejemplo es la primera fila de `pruebas_especificas_2015.csv`, tal como quedó guardada:

```
{
  "_id": "2015|949d241ac10256edf113|1|MATEMATICA",
  "aspirante_id": "949d241ac10256edf113",
  "correlativo_original": "949d241ac10256edf11339a303c72a50",
  "anio_ingreso": 2015,
  "oportunidad": 1,
  "materia": "MATEMATICA",
  "fecha_asignacion": ISODate("2014-08-23T00:00:00.000Z"),
  "anio_mes": "2014-08",
  "mes": 8,
  "aprobada": false,
  "carrera": null,
  "sexo": "M",
  "anio_nacimiento": null,
  "edad": null,
  "departamento": "GUATEMALA",
  "municipio": "GUATEMALA",
  "zona": 1,
  "tipo_institucion": "PRIVADO",
  "advertencias": [
    "SIN_ANIO_NACIMIENTO",
    "SIN_CARRERA"
  ],
  "origen": {
    "archivo": "pruebas_especificas_2015.csv",
    "linea": 2
  }
}
```


Los atributos del aspirante (sexo, geografía, carrera) se copian a cada evaluación a propósito: permite filtrar y agrupar evaluaciones sin consultar otra colección, igual que una tabla de hechos con sus dimensiones. Las fechas se guardan con tipo `Date`, no como texto.

5.3 Colección postulaciones

Un aspirante en un año de ingreso. Es la colección de las consultas «de aspirantes». El ejemplo es una postulación real aprobada, en la que Computación y Matemática se aprobaron en intentos distintos:


```
{
  "_id": "2020|002682dc0bebeba42770",
  "aspirante_id": "002682dc0bebeba42770",
  "anio_ingreso": 2020,
  "sexo": "M",
  "anio_nacimiento": 2001,
  "edad": 19,
  "departamento": "GUATEMALA",
  "municipio": "GUATEMALA",
  "zona": 12,
  "tipo_institucion": "PRIVADO",
  "carreras": [
    "INGENIERIA CIVIL"
  ],
  "materias": [
    {
      "materia": "MATEMATICA",
      "aprobada": true,
      "intentos": 2,
      "intento_aprobacion": 2,
      "fecha_primera_aprobacion": ISODate("2019-09-30T00:00:00.000Z"),
      "oportunidades": [
        1,
        2
      ]
    }
  ],
  "primer_intento": {
    "oportunidad": 1,
    "fecha": ISODate("2019-09-02T00:00:00.000Z"),
    "reprobo": true,
    "carreras": [
      "INGENIERIA CIVIL"
    ],
    "resultados": [
      {
        "materia": "MATEMATICA",
        "aprobada": false
```

```
      }
    ]
  },
  "n_evaluaciones": 2,
  "estado": "APROBADO",
  "aprobado": true,
  "anio_habilitacion": 2020,
  "fecha_habilitacion": ISODate("2019-09-30T00:00:00.000Z"),
  "aprobo_ambas_en_el_anio": false,
  "advertencias": []
}
```

Tabla 28. Campos principales de postulaciones

Campo	Notas
`sexo`, `edad`, geografía, `tipo_institucion`	Son constantes dentro de una postulación (se verificó), por eso van como campos simples
`carreras`	Arreglo de carreras distintas y no nulas. Con `$unwind` se agrupa por carrera
`materias`	Nivel 2 de aprobación. Es un **arreglo de subdocumentos** y no un objeto con claves `MATEMATICA`/`COMPUTACION`, para poder recorrerlo con `$unwind`
`primer_intento`	Evaluaciones de la menor oportunidad del año; `reprobo` indica si falló alguna materia
`estado` y `aprobado`	Indicador **principal** (c): habilitado con vigencia de 2 años por materia
`aprobo_ambas_en_el_anio`	Indicador **complementario** (a)
`anio_habilitacion`, `fecha_habilitacion`	Cuándo se habilitó la persona (nulo si nunca)

5.4 Colección aspirantes

Una persona con todo su historial. Es la colección de las consultas 1, 11, 12, 15 y 17 y del buscador por aspirante. Sus atributos (tipo de institución, procedencia) son los de su postulación más reciente con dato. El ejemplo abrevia el historial (7 evaluaciones en total):

```
{
  "_id": "002682dc0bebeba42770",
  "correlativo_original": "002682dc0bebeba427703d33aae5cb14",
  "sexo": "M",
  "anio_nacimiento": 2001,
  "tipo_institucion": "PRIVADO",
  "departamento": "GUATEMALA",
  "municipio": "GUATEMALA",
  "anios_ingreso": [
    2019,
    2020
  ],
  "habilitacion": {
    "fecha": ISODate("2019-09-30T00:00:00.000Z"),
    "anio": 2020
  },
  "historial": [
    {
      "anio_ingreso": 2019,
      "oportunidad": 2,
      "fecha": ISODate("2018-09-26T00:00:00.000Z"),
      "materia": "COMPUTACION",
      "aprobada": false,
      "carrera": "INGENIERIA CIVIL"
    },
    {
      "anio_ingreso": 2019,
      "oportunidad": 2,
      "fecha": ISODate("2018-09-26T00:00:00.000Z"),
      "materia": "MATEMATICA",
      "aprobada": false,
      "carrera": "INGENIERIA CIVIL"
    },
    "... 5 evaluaciones más"
  ],
  "advertencias": []
}
```

5.5 Colecciones resumen_carrera, cargas y rechazados

- **`resumen_carrera`** la genera la consulta 6 con un pipeline que termina en `$merge`: una fila por carrera con postulaciones, aspirantes distintos, aprobados con ambos indicadores, tasa de aprobación, edad promedio y el desglose por año.
- **`cargas`** tiene un documento por archivo procesado: nombre, leídos, cargados, rechazados, omitidos, tiempo, estado, rechazos por causa, advertencias por tipo y el resultado de la sincronización con Neo4j.
- **`rechazados`** tiene un documento por fila rechazada: la carga a la que pertenece, el archivo, la línea, la causa, un detalle y los valores originales de la fila.

5.6 Índices

Todos se crean al iniciar el backend y antes de cada carga con `createIndex`, que es idempotente. Se justifican con las consultas reales.

Tabla 29. Índices de MongoDB

Colección	Índice	Para qué
`evaluaciones`	`_id` (clave natural)	Unicidad; omitir duplicados al recargar
	`{aspirante_id, anio_ingreso, oportunidad}`	Reconstruir las derivadas leyendo por aspirante en orden
	`{materia, aprobada}`	Consultas 2 y 4
	`{anio_ingreso, aprobada}`	Consulta 20

Colección	Índice	Para qué
	`{tipo_institucion, anio_mes, materia}`	Consulta 9
`postulaciones`	`{aspirante_id}`	Unir con el historial de una persona
	`{anio_ingreso, aprobado}`	Consultas 3, 13, 18 y filtros por año
	`{carreras}` (multiclave)	Filtro por carrera
	`{departamento, municipio}`	Consultas 8 y 19 y el buscador
	`{edad}`	Consultas 10 y 13
`cargas`	`{iniciada_en: -1}`	Listar las cargas recientes
`rechazados`	`{carga_id, causa}`	Ver los rechazos de una carga, filtrados por causa

6. Modelo de Neo4j

Neo4j guarda las **relaciones** entre aspirantes, carreras, materias, instituciones y lugares. MongoDB responde «cuántos» y «qué porcentaje»; Neo4j responde «cómo se conectan».

6.1 Conceptos básicos

Tabla 30. Vocabulario de grafos

Término	Qué es
Nodo	Una «cosa» del grafo (un aspirante, una carrera). Tiene una **etiqueta** que indica su tipo.
Relación	Una conexión con **tipo y dirección** entre dos nodos, por ejemplo `(postulación)-[:ASPIRA_A]->(carrera)`.
Propiedad	Un dato guardado en un nodo o en una relación (`anio_ingreso: 2019`).
Cypher	El lenguaje de consultas. Los patrones se dibujan con paréntesis para los nodos y corchetes para las relaciones.
`MERGE`	«Crea el nodo o la relación solo si no existe»: es lo que evita duplicados.

6.2 Esquema

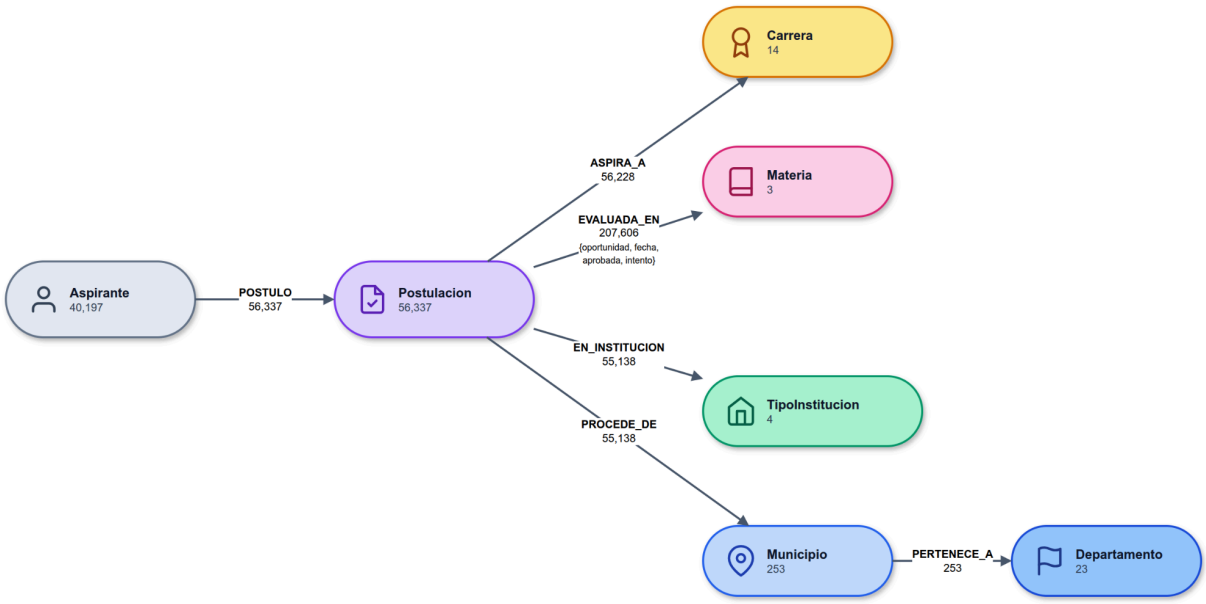


Figura 4. *Esquema del grafo: nodos, relaciones y cantidades*

Una **persona** (`Aspirante`) tiene una **postulación** por cada año de ingreso en que se presentó. Cada postulación apunta a la carrera a la que aspira, al municipio y tipo de institución de donde procede, y a las materias en las que fue evaluada.

Tabla 31. Nodos (96,831 en total)

Etiqueta	Cantidad	Clave única	Propiedades
`Aspirante`	40,197	`id`	`sexo`, `anio_nacimiento`
`Postulacion`	56,337	`id` = año|aspirante	`anio_ingreso`, `edad`, `zona`, `estado`, `aprobado`, `aprobo_ambas_en_el_anio`, `anio_habilitacion`, `n_evaluaciones`
`Carrera`	14	`nombre`	`ambigua` (solo en «Mecánica, sin precisar»)
`Materia`	3	`nombre`	—
`TipoInstitucion`	4	`nombre`	—
`Departamento`	23	`nombre`	22 departamentos y `EXTRANJERO`
`Municipio`	253	`id` = departamento|municipio	`nombre`, `departamento`

Tabla 32. Relaciones (430,700 en total)

Relación	Cantidad	Propiedades	Notas
`(Aspirante)-[:POSTULO]->(Postulacion)`	56,337	—	Una por año de ingreso
`(Postulacion)-[:ASPIRA_A]->(Carrera)`	56,228	—	Una por carrera distinta; no existe si la carrera es nula (6,146 postulaciones)
`(Postulacion)-[:EVALUADA_EN]->(Materia)`	207,606	`oportunidad`, `fecha`, `aprobada`, `intento`, `carrera`	**Una por evaluación.** Los reintentos se distinguen por `oportunidad`
`(Postulacion)-[:PROCEDE_DE]->(Municipio)`	55,138	—	No existe si la institución es `SIN DATOS`
`(Postulacion)-[:EN_INSTITUCION]->(TipoInstitucion)`	55,138	—	Igual
`(Municipio)-[:PERTENECE_A]->(Departamento)`	253	—	Jerarquía geográfica

6.3 Decisiones de diseño

1. **La postulación es un nodo propio.** Casi todo lo que se analiza cambia de un año a otro (carrera, edad, estado). Con la postulación como nodo intermedio el año es una propiedad, filtrar por año es una condición simple y se puede recorrer la historia de una persona de un año al siguiente. Es la misma idea que la colección `postulaciones` de MongoDB.
2. **Cada evaluación es una relación con propiedades, no un nodo.** Una evaluación solo conecta una postulación con una materia y no tiene nada colgando de ella; crear 207,606 nodos extra recargaría el grafo sin aportar nada.
3. **La geografía es una jerarquía Departamento – Municipio.** Hay nombres de municipio repetidos en departamentos distintos, por lo que el municipio se identifica con un `id` compuesto `DEPARTAMENTO|MUNICIPIO`. La zona de la capital es una propiedad de la postulación.
4. **El año de ingreso es una propiedad, no un nodo.** Un nodo `Anio` tendría hasta ~6,900 relaciones (un «súper nodo»), que ralentizan los recorridos y ensucian la visualización.

5. **Un dato ausente no genera relación.** `NINGUNO`, `SIN DATOS` y los valores nulos no crean nodos «vacíos» que mezclarían miles de aspirantes sin relación real entre sí.
6. **Cero duplicados por construcción.** Cada etiqueta tiene una restricción de unicidad y toda la carga usa `MERGE` sobre esa clave.

6.4 Restricciones e índices

```
CREATE CONSTRAINT aspirante_id        IF NOT EXISTS FOR (a:Aspirante)       REQUIRE a.id IS UNIQUE;
CREATE CONSTRAINT postulacion_id      IF NOT EXISTS FOR (p:Postulacion)     REQUIRE p.id IS UNIQUE;
CREATE CONSTRAINT carrera_nombre      IF NOT EXISTS FOR (c:Carrera)         REQUIRE c.nombre IS UNIQUE;
CREATE CONSTRAINT materia_nombre      IF NOT EXISTS FOR (m:Materia)         REQUIRE m.nombre IS UNIQUE;
CREATE CONSTRAINT tipo_nombre         IF NOT EXISTS FOR (t:TipoInstitucion) REQUIRE t.nombre IS UNIQUE;
CREATE CONSTRAINT departamento_nombre IF NOT EXISTS FOR (d:Departamento)    REQUIRE d.nombre IS UNIQUE;
CREATE CONSTRAINT municipio_id        IF NOT EXISTS FOR (m:Municipio)       REQUIRE m.id IS UNIQUE;

CREATE INDEX postulacion_anio     IF NOT EXISTS FOR (p:Postulacion) ON (p.anio_ingreso);
CREATE INDEX postulacion_aprobado IF NOT EXISTS FOR (p:Postulacion) ON (p.aprobado);
```

Las siete restricciones crean además sus propios índices. Los dos índices adicionales sirven para los filtros por año y por estado del explorador de grafos.

6.5 Carga del grafo

Tabla 33. Pasos de la sincronización con Neo4j

Paso	Qué hace	Lote
1	Crea las restricciones e índices	—
2	Catálogos: materias, tipos de institución, carreras, departamentos y municipios (con `PERTENECE_A`)	Todo (≈ 300 nodos)
3	Aspirantes, postulaciones y sus relaciones `POSTULO`, `ASPIRA_A`, `PROCEDE_DE` y `EN_INSTITUCION`	2,000
4	Relaciones `EVALUADA_EN` (con `oportunidad`, `fecha`, `aprobada` e `intento`)	5,000

Cada lote se envía con `UNWIND $filas AS f`, una sola consulta por lote en lugar de una por fila. El paso 4 usa:

```
UNWIND $filas AS e
MATCH (p:Postulacion {id: e.postulacion_id})
MATCH (m:Materia {nombre: e.materia})
MERGE (p)-[r:EVALUADA_EN {oportunidad: e.oportunidad}]->(m)
SET r.fecha = date(e.fecha), r.aprobada = e.aprobada, r.intento = e.intento, r.carrera = e.carrera
```

El driver devuelve contadores de lo que creó cada lote; se suman y se guardan en la bitácora de la carga. Se verificó que **volver a sincronizar no crea ningún nodo ni relación nuevos** (0 y 0).

8. Consultas

8.1 Criterios de diseño

1. **Cada consulta declara su unidad de análisis**: evaluación (una fila del CSV), postulación (un aspirante en un año) o persona (un aspirante en todos sus años). Mezclar unidades es la principal fuente de cifras que no cuadran entre sí.
2. **Cada consulta que habla de «aprobados» declara el nivel de aprobación** (sección 4.6): 1 evaluación, 2 materia del aspirante o 3 estado. Por eso la consulta 2 («aprobadas por materia») y la consulta 3 («aprobados por carrera») no se deben comparar directamente: cuentan cosas distintas.
3. **Los filtros son opcionales y se aplican al principio del pipeline** (`$match` antes de `$group`), para que MongoDB use los índices y procese menos documentos.
4. **Las consultas leen colecciones ya derivadas.** El trabajo costoso (estado de aprobación, primer intento) se hizo una vez en el ETL; las consultas solo agrupan y cuentan.
5. **Cada consulta tiene su tipo de gráfica** y, si hay muchas categorías, un límite explícito (por ejemplo, los 200 municipios con más aprobados) para que el resultado se pueda leer.

8.2 Consultas de MongoDB

Tabla 41. Las 20 consultas de MongoDB

N.º	Consulta	Colección	Unidad	Nivel de aprobación	Gráfica
1	Aspirantes por tipo de institución educativa	`aspirantes`	Persona	—	Pastel
2	Cantidad de evaluaciones aprobadas por materia	`evaluaciones`	Evaluación	1 · evaluación	Barras
3	Aspirantes aprobados por carrera y año de ingreso	`postulaciones`	Postulación	3 · estado (c)	Barras apiladas
4	Porcentaje de aprobación por materia	`evaluaciones`	Evaluación	1 · evaluación (y 2, informativo)	Barras agrupadas
5	Edad promedio por carrera	`postulaciones`	Postulación	—	Barras
6	Colección auxiliar resumen_carrera	`postulaciones`	Postulación	3 · estado (c) y (a)	Tabla
7	Edad promedio de aprobados por carrera y tipo de institución	`postulaciones`	Postulación	3 · estado (c)	Barras agrupadas
8	Distribución de aprobados por municipio y carrera	`postulaciones`	Postulación	3 · estado (c)	Tabla
9	Evaluaciones por mes y materia (instituciones públicas)	`evaluaciones`	Evaluación	—	Líneas
10	Top 5 de carreras más demandadas entre 16 y 18 años	`postulaciones`	Postulación	—	Barras
11	Historial de desempeño por aspirante	`aspirantes`	Persona	1 · evaluación (historial)	Tabla
12	Distribución por sexo y tipo de institución educativa	`aspirantes`	Persona	—	Barras apiladas
13	Tasa de aprobación por edad	`postulaciones`	Postulación	3 · estado (c)	Líneas

N.º	Consulta	Colección	Unidad	Nivel de aprobación	Gráfica
14	Número promedio de intentos por materia	`postulaciones`	Postulación	2 · materia del aspirante	Barras
15	Historial completo de un aspirante	`aspirantes`	Persona	1 · evaluación (historial)	Detalle
16	Carreras con más aspirantes reprobados en su primer intento	`postulaciones`	Postulación	Primer intento	Barras
17	Top 5 de municipios con más aspirantes	`aspirantes`	Persona	—	Barras
18	Tasa de aprobación por tipo de institución educativa	`postulaciones`	Postulación	3 · estado (c)	Barras
19	Carreras más demandadas por departamento	`postulaciones`	Postulación	—	Tabla
20	Evolución de la aprobación por año de ingreso	`evaluaciones`	Evaluación	1 · evaluación	Barras apiladas

Las 20 consultas se ejecutaron sobre los 207,606 documentos de `evaluaciones` y los 56,337 de `postulaciones`. El tiempo de ejecución fue de entre 2 y 571 ms (la más lenta, la consulta 4, porque calcula el porcentaje de aprobación por materia en dos niveles), medidos en un equipo de desarrollo, con los índices de la sección 5.6.

8.3 Consultas de Cypher

Tabla 42. Las 6 consultas de Cypher

Id	Consulta	Qué muestra del lenguaje	Gráfica
G1	Carreras más demandadas desde un departamento	Recorrido de 3 relaciones; Filtro por propiedad de nodo; Agregación con count	Barras
G2	Pares de carreras que comparten aspirantes	Patrón con un nodo intermedio compartido; Dos relaciones del mismo tipo; Filtro opcional por propiedad	Tabla
G3	Aspirantes que cambian de carrera de un año al siguiente	Recorrido que cruza años a través del aspirante; Aritmética sobre propiedades; Negación de patrones	Tabla
G4	Aprobaron Matemática tras varios intentos, por tipo de institución	Propiedades en relaciones; Filtro sobre propiedades de la relación; Unión con un segundo recorrido	Barras agrupadas
G5	Flujo de aprobados: de qué departamentos a qué carreras	Camino de varias relaciones; Propiedad de nodo dentro del patrón; Resultado como aristas ponderadas	Tabla
G6	Aspirantes que postulan varios años y cuántos terminan aprobando	Conteo de relaciones por nodo; Agregación en dos niveles con WITH; Comparación entre grupos	Barras

Los tiempos medidos estuvieron entre 97 y 665 ms (la más lenta, la G3, que compara postulaciones de años consecutivos de la misma persona). Todas son de solo lectura, y una prueba automática lo comprueba.

9. Análisis de resultados

Este capítulo interpreta lo que muestran las 26 consultas. Cada afirmación indica de qué consulta sale. Los datos permiten **describir** lo que ocurrió; las explicaciones de *por qué* ocurrió se presentan como hipótesis cuando los datos no las demuestran.

9.1 Cómo leer los resultados

Una misma pregunta («¿qué porcentaje aprueba?») tiene respuestas distintas según qué se cuente. Las cifras de este capítulo usan siempre una de estas tres medidas (sección 4.6):

Tabla 49. Tres formas de medir la aprobación

Nivel	Qué cuenta	Cifras	Consulta
1 · Evaluación	Cada examen aprobado sobre cada examen presentado	Matemática 8.8 %, Computación 22.4 %	2, 4, 20
2 · Materia del aspirante	Aspirantes que aprobaron la materia en algún intento del año	Matemática 19.5 %, Computación 41.3 %	4, 14
3 · Estado (indicador principal)	Postulaciones habilitadas: Matemática y Computación aprobadas con vigencia de 2 años	8,471 postulaciones (15.0 %)	3, 6, 7, 8, 13, 18

No se deben comparar cifras de niveles distintos. Que «Matemática se apruebe en el 8.8 % de las evaluaciones» y que «el 19.5 % de los aspirantes la apruebe» no se contradicen: quien reprueba vuelve a presentarse, y cada reintento fallido baja el primer porcentaje sin afectar el segundo.

9.2 Panorama general

Tabla 50. Cifras generales de los 10 años

Medida	Valor
Filas leídas de los 10 archivos	208,453
Evaluaciones cargadas	207,606 (847 duplicados descartados, 0.4 %)
Personas distintas	40,197
Postulaciones (persona-año)	56,337
Postulaciones habilitadas (indicador principal)	8,471 (15.0 %)
Solo lo aprobado dentro del propio año (complementario)	7,011 (12.4 %)
Personas que llegaron a habilitarse alguna vez	8,471 de 40,197 (21.1 %)
Edad promedio	20.63 años (6,689 postulaciones sin edad)

- **Aprueba una minoría.** De cada 100 postulaciones, 15 terminan habilitadas. Visto por personas el porcentaje es mayor (21.1 %), porque una persona puede postular varios años y basta que se habilite una vez.
- **La mayoría participa un solo año:** 28,229 personas (70.2 %), 9,105 dos años (22.7 %) y 2,863 tres o más (7.1 %) (consulta G6).

- **El estado «parcial» es grande.** 20,928 postulaciones (37 %) tienen alguna materia aprobada y no llegaron a habilitarse (en este estado no se mira si esa aprobación seguía vigente).

9.3 Evolución por año

Tabla 51. Resultados por año de ingreso

Año	Postulaciones	Habilitadas (c)	Tasa (c)	Tasa (a)	Evaluaciones	Evaluaciones por postulación
2015	6,148	910	14.8 %	14.8 %	20,266	3.30
2016	6,428	908	14.1 %	9.8 %	21,987	3.42
2017	5,976	1,012	16.9 %	14.0 %	21,398	3.58
2018	5,738	1,071	18.7 %	15.6 %	20,678	3.60
2019	5,310	1,216	22.9 %	16.1 %	17,835	3.36
2020	4,997	986	19.7 %	15.5 %	17,665	3.54
2021	6,861	449	6.5 %	5.2 %	30,642	4.47
2022	6,380	447	7.0 %	6.1 %	24,231	3.80
2023	4,638	765	16.5 %	15.4 %	19,047	4.11
2024	3,861	707	18.3 %	16.9 %	13,857	3.59

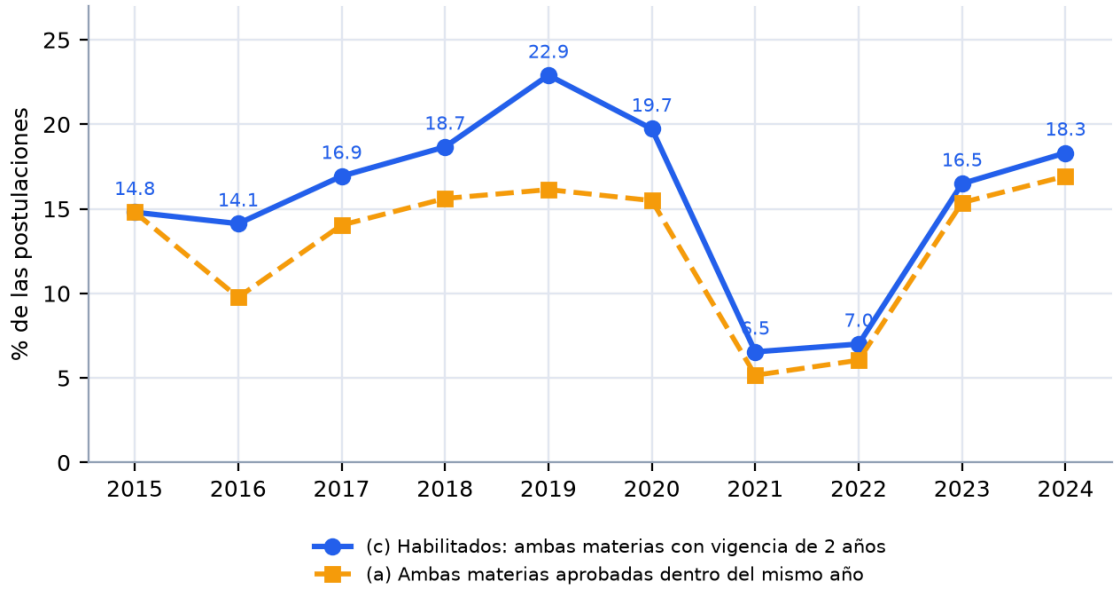


Figura 5. *Porcentaje de postulaciones habilitadas por año, con el indicador principal (c) y el complementario (a)*

Entre 2015 y 2020 la tasa se mantuvo entre 14 % y 23 %, con un máximo en 2019 (22.9 %). En **2021 y 2022 cae a 6.5 % y 7.0 %**, y en 2023 y 2024 se recupera a 16.5 % y 18.3 %.

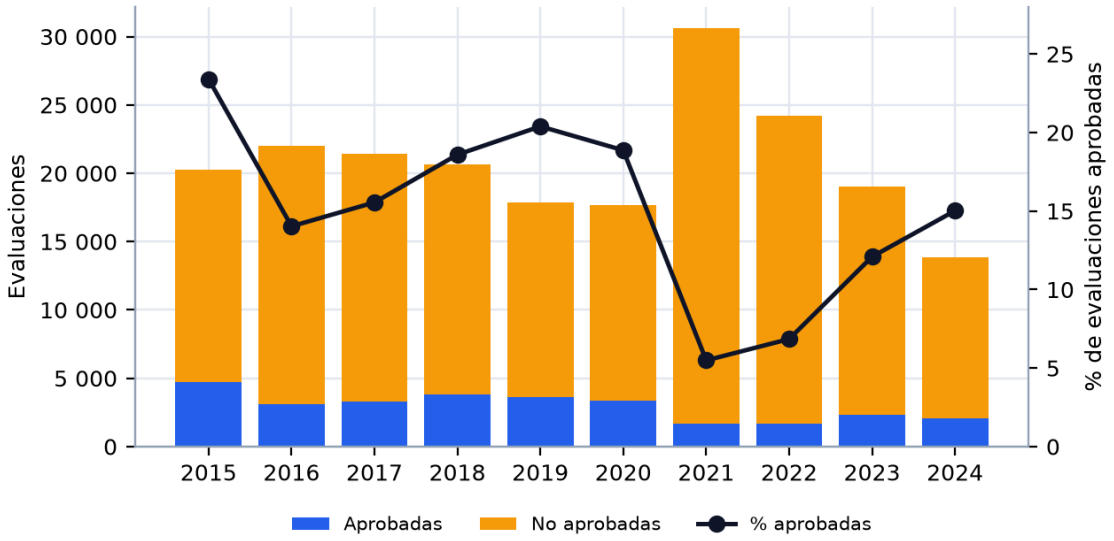


Figura 6. *Evaluaciones por año y porcentaje aprobado*

La caída de 2021 tiene dos rasgos a la vez: es el año con **más postulaciones** (6,861) y con **más evaluaciones** (30,642, contra 17,700 a 22,000 en 2015–2020), es decir, 4.47 evaluaciones por postulación frente a 3.3 en 2015. Hubo más intentos y menos aprobaciones.

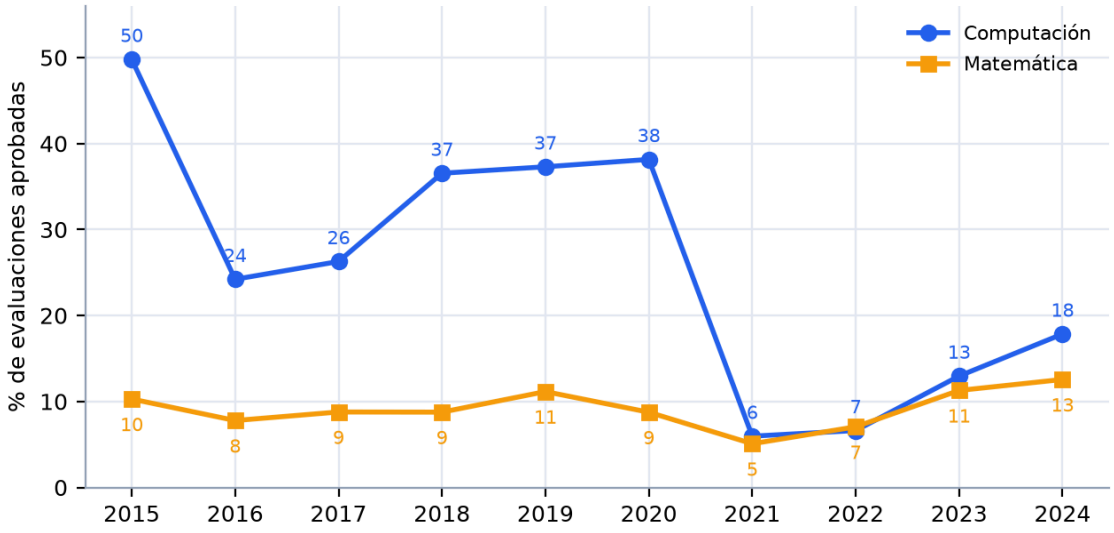


Figura 7. *Porcentaje de evaluaciones aprobadas por materia y año*

La caída se concentra en **Computación**: de 38.2 % de las evaluaciones aprobadas en 2020 pasa a 6.0 % en 2021 (y venía de 49.8 % en 2015). Matemática se movió menos (8.8 % → 5.1 %). Los datos no explican la causa. Una hipótesis razonable es un cambio en la forma de aplicar o calificar la prueba en esos años (coinciden con la pandemia), pero **es una hipótesis que habría que confirmar con la Facultad**, no una conclusión de este trabajo.

Efecto de los bordes del período. 2015 es el primer año de los datos: no se ven aprobaciones anteriores, por lo que su tasa (c) es igual a la (a). Del mismo modo, quienes postularon en 2023–2024 podrían habilitarse después de 2024, así que el límite de los datos subestima un poco los últimos años.

9.4 Por materia

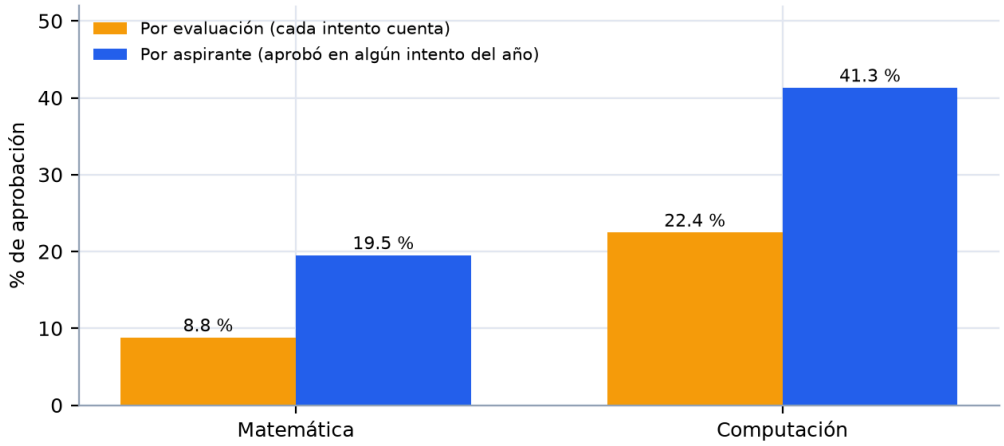


Figura 8. Porcentaje de aprobación por materia, a nivel de evaluación y a nivel de aspirante

- **Matemática es el cuello de botella.** Se aprueba en el 8.8 % de las evaluaciones (19.5 % de los aspirantes) frente al 22.4 % (41.3 %) de Computación. La proporción de aspirantes que aprueba Computación es más del doble que la de Matemática.
- **Matemática exige más intentos:** 2.22 intentos en promedio por postulación frente a 1.84 en Computación; el máximo en ambas es 5 (consulta 14).
- **Casi todas las postulaciones presentan Matemática (99.2 %), pero solo el 80.7 % presenta Computación** (55,865 y 45,476 de 56,337). Los datos no dicen por qué el resto no la presentó.
- `MATEMATICA ITUGS` es una materia aparte con solo 22 evaluaciones y 14 aspirantes; no se aprobó ninguna. Se mantiene separada para no mezclarla.

9.5 Reintentos y perseverancia

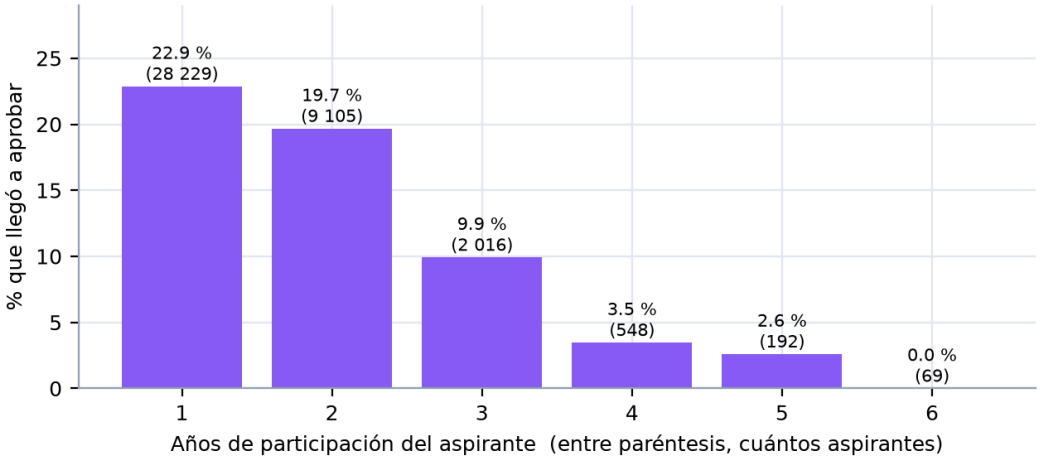


Figura 9. Porcentaje de aspirantes que llegó a aprobar según los años que participó

- **Casi nadie aprueba todo en el primer intento.** Entre el 95.8 % y el 98.0 % de los aspirantes de cada carrera de ingeniería reprobó al menos una materia en su primer intento (consulta 16;

en las licenciaturas, 79 % y 88 %). Como el valor es casi igual en las carreras con más demanda, **no distingue** entre ellas.

- **Aprobar suele requerir insistir:** entre quienes aprobaron Matemática en instituciones privadas, el 37.3 % lo hizo en el tercer intento o después (G4: 3,593 de 9,631).
- **Con más años de participación, menos aprueban:** 22.9 % de quienes participan un año, 19.7 % de dos, 9.9 % de tres, 3.5 % de cuatro, 2.6 % de cinco y ninguno de los 107 que participaron seis años o más (G6).

9.6 Tipo de institución

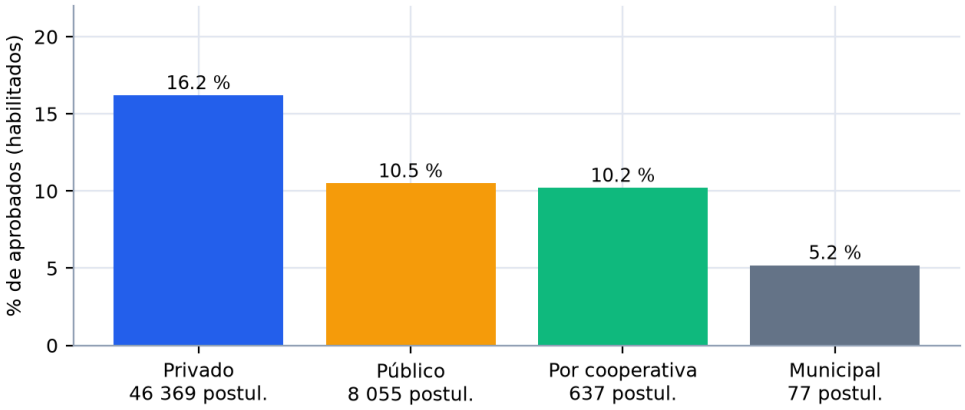


Figura 10. *Porcentaje de postulaciones habilitadas por tipo de institución educativa*

- **La gran mayoría proviene de institutos privados:** 33,477 de 39,279 personas con institución conocida (85.2 %) (consulta 1).
- **Los privados aprueban más:** 16.2 % (7,513 de 46,369 postulaciones) frente a 10.5 % de los públicos (845 de 8,055). Cooperativa (10.2 %) y municipal (5.2 %) tienen pocos casos (637 y 77 postulaciones): sus porcentajes son poco confiables (consulta 18).
- **Nueve de cada diez habilitados vienen de un instituto privado** (7,513 de los 8,427 habilitados con institución conocida).

La diferencia entre privados y públicos es una **asociación**, no una explicación: puede reflejar la preparación previa, pero también otras diferencias entre los dos grupos (zona, nivel socioeconómico) que los datos no incluyen.

9.7 Edad y sexo

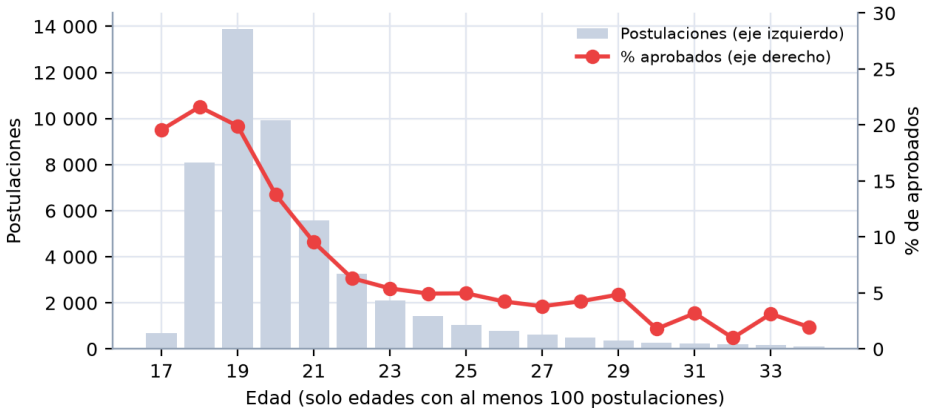


Figura 11. *Postulaciones y porcentaje de habilitados por edad*

- **La demanda se concentra en 18 a 20 años:** casi dos de cada tres postulaciones con edad conocida (31,897 de 49,648). La edad promedio es 20.6 años.
- **La tasa de aprobación baja con la edad:** 21.6 % a los 18 años, 19.9 % a los 19, 13.8 % a los 20, 9.5 % a los 21 y 6.3 % a los 22. A partir de los 25 años ronda entre 4 % y 5 % y sigue bajando (consulta 13).
- **Tres de cada cuatro aspirantes son hombres:** 30,718 hombres (76.4 %), 9,419 mujeres (23.4 %) y 60 sin registro.

La edad también refleja cuántas veces se ha intentado: quien tiene 24 años probablemente ya participó antes (sección 10.5).

9.8 Carreras y geografía

Tabla 52. Resumen por carrera (consulta 6)

Carrera	Postulaciones	Habilitadas	Tasa	Edad prom.	Años
Ing. EN CIENCIAS Y SISTEMAS	18,588	2,817	15.2 %	20.4	2015–2024
Ing. CIVIL	9,062	1,051	11.6 %	20.9	2016–2024
Ing. INDUSTRIAL	8,760	1,097	12.5 %	20.9	2016–2024
Ing. QUIMICA	4,816	902	18.7 %	19.9	2016–2024
Ing. ELECTRONICA	3,858	692	17.9 %	20.7	2016–2024
Ing. MECANICA	2,876	356	12.4 %	21.0	2016–2024
Ing. ELECTRICA	2,376	383	16.1 %	21.2	2016–2024
Ing. MECANICA INDUSTRIAL	1,965	352	17.9 %	20.5	2016–2021
Ing. MECANICA ELECTRICA	1,573	328	20.9 %	20.9	2016–2021
Ing. MECANICA (sin precisar)	1,128	181	16.1 %	20.9	2022–2024
Ing. AMBIENTAL	946	120	12.7 %	20.7	2016–2024
Lic. FISICA APLICADA	193	23	11.9 %	21.0	2016–2019
Lic. MATEMATICA APLICADA	73	5	6.8 %	21.2	2016–2019
ITUGS	14	0	0.0 %	25.0	2021–2022

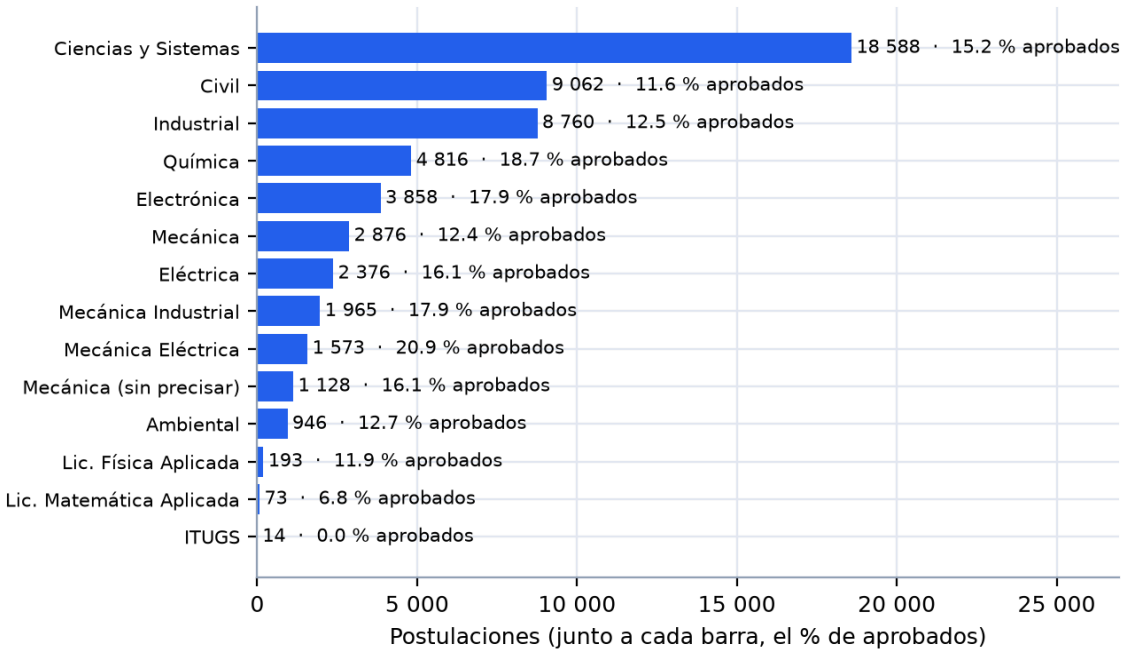


Figura 12. *Carreras con más postulaciones y su porcentaje de habilitados*

- **La demanda está muy concentrada:** Ciencias y Sistemas, Civil e Industrial suman el 65 % de las postulaciones a carrera (36,410 de 56,228). Ciencias y Sistemas por sí sola, el 33 %.
- **Ciencias y Sistemas lidera en 17 de los 23 departamentos y Civil en los otros 6** (consulta 19); por ejemplo, en Alta Verapaz es Civil (258) antes que Sistemas (166).
- **El departamento de Guatemala aporta el 76.7 % de las personas** (30,141 de 39,279 con departamento conocido). Las tres mayores procedencias son municipios de ese departamento: Guatemala (19,024), Mixco (4,905) y Villa Nueva (3,134) (consulta 17).
- **Entre las carreras con más de 1,000 postulaciones, la tasa va de 11.6 % (Civil) a 20.9 % (Mecánica Eléctrica).** Todas las carreras se enfrentan a las mismas dos pruebas, así que estas diferencias dicen **quién elige cada carrera**, no qué tan difícil es entrar a ella.

9.9 Relaciones entre carreras (Neo4j)

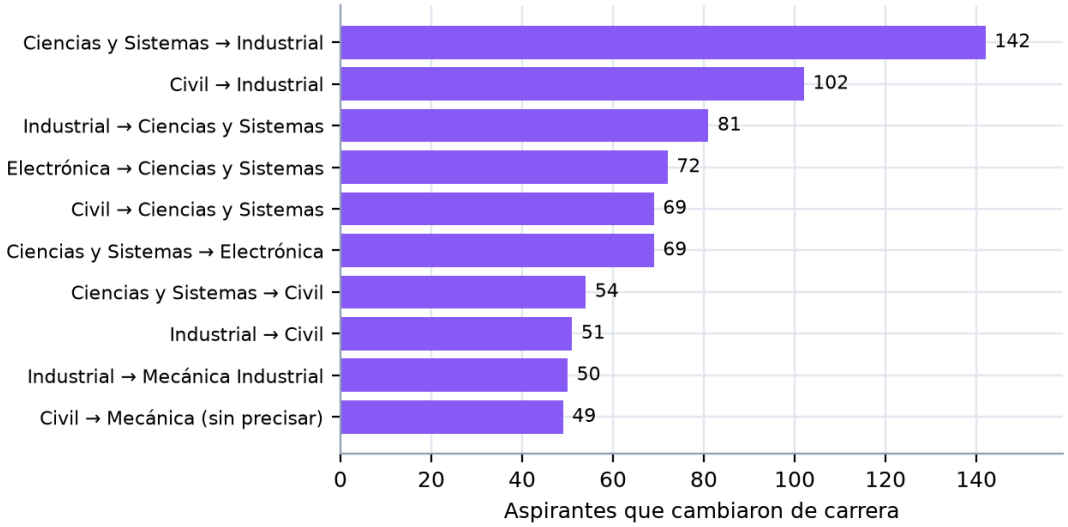


Figura 13. *Aspirantes que cambian de carrera de un año al siguiente (G3)*

- **Carreras que se eligen juntas (G2):** Civil + Ciencias y Sistemas es el par más frecuente (892 postulaciones), seguido de Civil + Industrial (654) y Ciencias y Sistemas + Industrial (446). También aparecen pares «de la misma familia», como Industrial + Mecánica Industrial (297) o Eléctrica + Electrónica (236).
- **Cambios de carrera (G3):** el cambio más común es de Ciencias y Sistemas a Industrial (142 aspirantes), seguido de Civil a Industrial (102). Industrial es el principal destino de quienes cambian.
- **Origen de los aprobados (G5):** del departamento de Guatemala salen 2,338 aprobados hacia Ciencias y Sistemas, 911 a Industrial y 760 a Civil.

Estas preguntas se contestan con un recorrido por el grafo; en MongoDB serían una unión del documento consigo mismo (sección 11.2).

9.10 Calidad de los datos y su efecto en el análisis

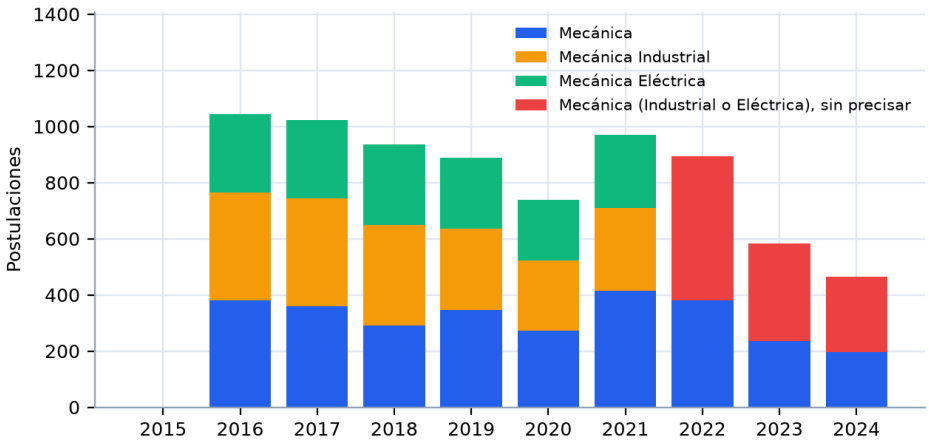


Figura 14. *Postulaciones a las tres carreras de Mecánica por año: en 2022 las dos específicas dejan de usarse*

- **Las carreras de Mecánica cambian de nombre en 2022.** Desde ese año los registros de Mecánica Industrial y Mecánica Eléctrica pasan a ser «Mecánica (Industrial o Eléctrica, sin precisar)»: 513, 347 y 268 postulaciones en 2022, 2023 y 2024. Se conservó como una carrera aparte, marcada como ambigua, en vez de repartirla sin base. **No se deben sumar ni comparar como una serie continua.**
- **El año 2015 no trae carrera ni edad** en 6,146 y 6,689 postulaciones. Las consultas por carrera y por edad no incluyen esos casos, y aun así documentan cuántos son.
- **Hay carreras con historia corta:** Física y Matemática Aplicada solo aparecen entre 2016 y 2019; ITUGS, con 14 postulaciones, entre 2021 y 2022.
- **Los 847 duplicados (0.4 %)** vienen de 818 grupos de filas repetidas: 805 idénticas y 13 que difieren solo en la fecha o la carrera, sin resultados contradictorios. Se conservó la de fecha más temprana (sección 4.5), así que no alteran las conclusiones.

11. Conclusiones

11.1 Sobre los datos

1. **La aprobación es la excepción, no la regla.** El 15.0 % de las postulaciones se habilita (8,471 de 56,337), y más del 95 % de los aspirantes a ingeniería reprueba al menos una materia en su primer intento. La habilitación suele lograrse tras varios intentos: alrededor del 38 % de las aprobaciones de Matemática ocurre en el tercer intento o después (4,080 de 10,831, consulta G4).
2. **Matemática es el principal obstáculo.** Se aprueba en menos de una de cada diez evaluaciones, con un promedio de 2.2 intentos, frente a 1.8 de Computación.
3. **El período 2021–2022 es un quiebre en los datos.** La tasa baja de entre 14 % y 23 % a 6.5 % y 7.0 %, por una caída de la aprobación de Computación de 38 % a 6 %. Se recupera en 2023–2024. La causa no está en los datos.
4. **La demanda y los resultados dependen del perfil del aspirante:** instituto privado (85 % de las personas), departamento de Guatemala (77 %), hombre (76 %), 18 a 20 años. Los privados y los más jóvenes aprueban más; son asociaciones, no causas.
5. **La definición de «aprobado» cambia el resultado:** 7,011 postulaciones con el criterio más estricto (dentro del propio año), 8,471 con el elegido, y entre 7,884 (vigencia de 1 año) y 8,659 (sin límite) según la vigencia. Por eso la definición se fundamentó con el reglamento y se documentó (sección 4.6).
6. **La calidad de los datos limita lo que se puede concluir:** carreras con nombres truncados o ambiguos, geografía incompleta desde 2022, y falta de carrera y edad en 2015. Las reglas del ETL las corrigen cuando la corrección es segura y las marcan cuando no.

11.2 Sobre MongoDB y Neo4j trabajando juntos

El enunciado exige las dos bases de datos. La experiencia de construirlo permite compararlas con ejemplos concretos del proyecto:

Tabla 54. Comparación de las dos bases de datos en este proyecto

	MongoDB	Neo4j
Lo que mejor hizo	Agrupar y contar: porcentajes, promedios y series por año, con pipelines de pocas etapas. Filtrar con muchos criterios y paginar	Seguir conexiones: «carreras que comparten aspirantes» (G2) o «de qué carrera a qué carrera se mueve la gente» (G3) son un solo patrón
Modelado	Pide decidir qué embeber y qué derivar. El estado de aprobación se calcula una vez y se guarda; las consultas quedan simples	Pide decidir qué es un nodo y qué una relación (la postulación como nodo y la evaluación como relación)
Visualización	Datos para gráficas y tablas	El grafo es la visualización: el mapa de demanda y la red de un aspirante se obtienen directamente
Lo que cuesta pedirle lo del otro	G3 (cambios de carrera) exige agrupar el historial de cada persona y comparar años consecutivos; es posible, pero largo (así se recalculó para verificar)	Sumar aprobados por carrera y año (consulta 3) es posible, pero recorre muchos nodos para algo que MongoDB resuelve con un `$group`

	MongoDB	Neo4j
Escala de este proyecto	Las 20 consultas tardan entre 2 y 571 ms	Las 6 consultas tardan entre 97 y 665 ms

Lo que cuesta usar las dos. Hay que mantener los datos sincronizados: Neo4j se llena desde MongoDB al terminar cada carga, y si falla esa segunda etapa los dos quedan desalineados hasta reconstruir (por eso existe `POST /api/etl/reconstruir`). Además se duplica el almacenamiento, el código de acceso y los modos de falla. Para un conjunto de datos de este tamaño, MongoDB podría contestar también las preguntas de relaciones con más esfuerzo; el valor de Neo4j aquí es que esas preguntas se expresan de forma natural y que permite visualizar la red.

Conclusión: las dos bases son complementarias si se asigna a cada una lo que mejor hace: MongoDB para la analítica y el buscador, Neo4j para las relaciones y la visualización. Mantenerlas alineadas se logró haciendo que **todo salga de una única fuente** (las colecciones derivadas de MongoDB) y que la carga a Neo4j sea idempotente, y se comprobó con 11 verificaciones cruzadas.

11.3 Limitaciones

- **Del alcance de los datos:** son solo las pruebas específicas; no se sabe quién terminó siendo admitido a una carrera ni los resultados de las pruebas básicas. «Aprobado» significa *habilitado para continuar*, no *admitido*.
- **De la definición de aprobación:** el criterio principal (vigencia de 2 años por materia desde la fecha de asignación) se sustenta en el reglamento y en una constancia, pero no se pudo confirmar con la Facultad que el criterio con que los CSV se generaron sea idéntico. Por eso se calculan y conservan los otros indicadores.
- **De los bordes del período:** 2015 no tiene historial previo y 2023–2024 no tienen seguimiento posterior; esto subestima levemente las habilitaciones de esos años.
- **De la identificación de personas:** desde 2022 el identificador viene truncado, por lo que se usan los primeros 20 caracteres de todos. Con los 29,418 identificadores completos de 2015–2021 no hay colisiones a esa longitud, pero no se puede comprobar para los truncados: dos personas distintas con el mismo prefijo se verían como una sola.
- **Del sistema:** no tiene usuarios ni autenticación (es una aplicación académica local); las consultas se ejecutaron con un solo usuario; la interfaz no tiene pruebas automáticas; el borrado y la carga son operaciones de administración sin control de acceso.

Anexo A. Consultas de MongoDB

Las 20 consultas, con el pipeline exacto que ejecuta el servidor y las primeras filas del resultado sobre los datos cargados. Los pipelines están escritos en la sintaxis de `mongosh`, de modo que se pueden copiar y pegar en MongoDB Compass o en la consola. Los parámetros opcionales (por ejemplo, el filtro por año) agregan un `$match` al inicio y aquí se muestran sin filtros.

Las mismas consultas, con el resultado completo, están en `docs/consultas-mongodb.md`, que se regenera con `npm run doc:consultas`.

A.1 Consultas 1 a 10

Consulta 1 · Aspirantes por tipo de institución educativa

Pregunta. Cantidad de aspirantes (personas distintas) agrupados por el tipo de institución de la que proceden.

Cómo funciona. La colección `aspirantes` tiene un documento por persona, así que contar documentos es contar personas. Se excluyen los aspirantes sin dato de institución (`SIN DATOS` pasó a `null` en el ETL).

Pipeline sobre `aspirantes`:

```
db.aspirantes.aggregate([
  {"$match":{"tipo_institucion":{"$ne":null}}},
  {"$group":{"_id":"$tipo_institucion","aspirantes":{"$sum":1}}},
  {"$sort":{"aspirantes":-1}},
  {"$project":{"_id":0,"tipo_institucion":"$_id","aspirantes":1}}
])
```

Resultado (41 ms · 4 filas):

tipo institucion	aspirantes
PRIVADO	33,477
PUBLICO	5,313
POR COOPERATIVA	434
MUNICIPAL	55

Consulta 2 · Cantidad de evaluaciones aprobadas por materia

Pregunta. Número de evaluaciones aprobadas, agrupadas por materia.

Cómo funciona. Es una consulta a nivel de EVALUACIÓN (una fila del CSV): se filtran las evaluaciones con `aprobada = true` y se cuentan por materia. Un aspirante que aprueba una materia en su tercer intento aporta una sola evaluación aprobada.

Tabla 56. Parámetros de la consulta 2

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `evaluaciones`:

```
db.evaluaciones.aggregate([
  {"$match":{"aprobada":true}},
  {"$group":{"_id":"$materia","aprobadas":{"$sum":1}}},
  {"$sort":{"aprobadas":-1}},
  {"$project":{"_id":0,"materia":"$_id","aprobadas":1}}
])
```

Resultado (107 ms · 2 filas):

materia	aprobadas
COMPUTACION	18,799
MATEMATICA	10,898

Consulta 3 · Aspirantes aprobados por carrera y año de ingreso

Pregunta. Cantidad de aspirantes aprobados (habilitados) agrupados por carrera objetivo y año de ingreso.

Cómo funciona. Usa `postulaciones` (un aspirante en un año). Se filtran las postulaciones con `aprobado = true` (indicador principal: Matemática y Computación aprobadas con vigencia de 2 años por materia). `$unwind` convierte el arreglo `carreras` en una fila por carrera, de modo que una postulación con dos carreras cuenta en ambas. Las postulaciones sin carrera (casi todas de 2015, donde la carrera era `NINGUNO`) no aparecen aquí.

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"aprobado":true}},
  {"$unwind":"$carreras"},
  {"$group":{"_id":{"carrera":"$carreras","anio_ingreso":"$anio_ingreso"},"aprobados":{"$sum":1}}},
  {"$sort":{"_id.anio_ingreso":1,"_id.carrera":1}},
  {"$project":{"_id":0,"anio_ingreso":"$_id.anio_ingreso","carrera":"$_id.carrera","aprobados":1}}
])
```

Resultado (44 ms · primeras 8 de 94 filas):

carrera	aprobados	anio ingreso
Ing. AMBIENTAL	14	2016
Ing. CIVIL	140	2016
Ing. ELECTRICA	40	2016
Ing. ELECTRONICA	92	2016
Ing. EN CIENCIAS Y SISTEMAS	325	2016
Ing. INDUSTRIAL	131	2016
Ing. MECANICA	40	2016
Ing. MECANICA ELECTRICA	49	2016

Consulta 4 · Porcentaje de aprobación por materia

Pregunta. Porcentaje de aprobación de cada materia, medido de dos maneras: por evaluación y por aspirante.

Cómo funciona. Principal: aprobadas / evaluaciones (nivel evaluación, el mismo criterio de la consulta 2). Como quien reprueba vuelve a intentarlo, los reintentos fallidos bajan este porcentaje.

Informativo: a nivel aspirante-materia-año (¿aprobó la materia en algún intento de ese año?), que se calcula con un `$lookup` a `postulaciones`. Es una medida mucho más alta y más cercana a la experiencia de una persona.

Tabla 57. Parámetros de la consulta 4

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `evaluaciones`:

```
db.evaluaciones.aggregate([
  {
    "$group": {"_id":"$materia","evaluaciones":{"$sum":1},"aprobadas":{"$sum":{"$cond":["$aprobada",1,0]}}}
  },
  {
    "$lookup": {
      "from": "postulaciones",
      "let": {"m":"$_id"},
      "pipeline": [
        {"$unwind":"$materias"},
        {"$match":{"$expr":{"$eq":["$materias.materia","$$m"]}}},
        {
          "$group": {
            "_id": null,
            "aspirantes": {"$sum":1},
            "aprobados": {"$sum":{"$cond":["$materias.aprobada",1,0]}}
          }
        }
      ],
      "as": "por_aspirante"
    }
  },
  {"$unwind":{"path":"$por_aspirante","preserveNullAndEmptyArrays":true}},
  {
    "$project": {
      "_id": 0,
      "materia": "$_id",
      "evaluaciones": 1,
      "aprobadas": 1,
      "porcentaje_evaluaciones": {"$round":[{"$multiply":[{"$divide":["$aprobadas","$evaluaciones"]},100]},2]},
      "aspirantes_materia": {"$ifNull":["$por_aspirante.aspirantes",0]},
      "aprobados_aspirante": {"$ifNull":["$por_aspirante.aprobados",0]},
      "porcentaje_aspirantes": {
        "$cond": [
          {"$gt":[{"$ifNull":["$por_aspirante.aspirantes",0]},0]},
          {
            "$round": [
              {
                "$multiply": [{"$divide":["$por_aspirante.aprobados","$por_aspirante.aspirantes"]},100]
              },
              2
            ]
          },
          null
        ]
      }
    }
  },
  {"$sort":{"evaluaciones":-1}}
])
```

Resultado (571 ms · 3 filas):

materia	evaluaciones	aprobadas	porcentaje evaluaciones	aspirantes materia	aprobados aspirante
MATEMATICA	123,830	10,898	8.8	55,865	10,896
COMPUTACION	83,754	18,799	22.45	45,476	18,783
MATEMATICA ITUGS	22	0	0	14	0

Consulta 5 · Edad promedio por carrera

Pregunta. Edad promedio de los aspirantes agrupados por carrera objetivo.

Cómo funciona. Usa `postulaciones`. La edad es `anio_ingreso - anio_nacimiento` y se calculó en el ETL (R-ED1); las postulaciones sin edad válida (año de nacimiento `SIN REGISTRO` o fuera de 13–70) se excluyen con `$match`. `$unwind` sobre `carreras` cuenta cada postulación en cada una de sus carreras.

Tabla 58. Parámetros de la consulta 5

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"edad":{"$ne":null}}},
  {"$unwind":"$carreras"},
  {"$group":{"_id":"$carreras","edad_promedio":{"$avg":"$edad"},"aspirantes":{"$sum":1}}},
  {"$sort":{"edad_promedio":1}},
  {
    "$project": {"_id":0,"carrera":"$_id","edad_promedio":{"$round":["$edad_promedio",2]},"aspirantes":1}
  }
])
```

Resultado (115 ms · primeras 8 de 14 filas):

carrera	aspirantes	edad promedio
Ing. QUIMICA	4,599	19.92
Ing. EN CIENCIAS Y SISTEMAS	17,911	20.35
Ing. MECANICA INDUSTRIAL	1,850	20.54
Ing. AMBIENTAL	900	20.73
Ing. ELECTRONICA	3,705	20.73
Ing. INDUSTRIAL	8,387	20.85
Ing. CIVIL	8,654	20.94
Ing. MECANICA (sin precisar)	1,127	20.94

Consulta 6 · Colección auxiliar resumen_carrera

Pregunta. Genera (o actualiza) la colección `resumen_carrera` con un resumen por carrera y muestra su contenido.

Cómo funciona. El pipeline agrupa `postulaciones` por carrera (con `$unwind` sobre `carreras`), calcula totales, tasas y edades promedio, y termina con `$merge`, que escribe el resultado en la colección `resumen_carrera` en vez de devolverlo. Al terminar, la consulta lee esa colección para mostrarla. Se vuelve a generar sola después de cada carga de datos.

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"carreras.0":{"$exists":true}}},
  {"$unwind":"$carreras"},
  {
    "$group": {
      "_id": {"carrera":"$carreras","anio":"$anio_ingreso"},
      "postulaciones": {"$sum":1},
```

```
      "aspirantes": {"$addToSet":"$aspirante_id"},
      "aprobados": {"$sum":{"$cond":["$aprobado",1,0]}},
      "aprobados_en_el_anio": {"$sum":{"$cond":["$aprobo_ambas_en_el_anio",1,0]}},
      "suma_edad": {"$sum":{"$ifNull":["$edad",0]}},
      "n_edad": {"$sum":{"$cond":[{"$gt":["$edad",null]},1,0]}},
      "suma_edad_aprobados": {"$sum":{"$cond":["$aprobado",{"$ifNull":["$edad",0]},0]}},
      "n_edad_aprobados": {"$sum":{"$cond":[{"$and":["$aprobado",{"$gt":["$edad",null]}]},1,0]}}
    }
  },
  {
    "$group": {
      "_id": "$_id.carrera",
      "postulaciones": {"$sum":"$postulaciones"},
      "aspirantes": {"$push":"$aspirantes"},
      "aprobados": {"$sum":"$aprobados"},
      "aprobados_en_el_anio": {"$sum":"$aprobados_en_el_anio"},
      "suma_edad": {"$sum":"$suma_edad"},
      "n_edad": {"$sum":"$n_edad"},
      "suma_edad_aprobados": {"$sum":"$suma_edad_aprobados"},
      "n_edad_aprobados": {"$sum":"$n_edad_aprobados"},
      "primer_anio": {"$min":"$_id.anio"},
      "ultimo_anio": {"$max":"$_id.anio"},
      "por_anio": {
        "$push": {"anio_ingreso":"$_id.anio","postulaciones":"$postulaciones","aprobados":"$aprobados"}
      }
    }
  },
  {
    "$project": {
      "carrera": "$_id",
      "postulaciones": 1,
      "aspirantes_distintos": {
        "$size": {
          "$reduce": {"input":"$aspirantes","initialValue":[],"in":{"$setUnion":["$$value","$$this"]}}
        }
      },
      "aprobados": 1,
      "tasa_aprobacion": {"$round":[{"$multiply":[{"$divide":["$aprobados","$postulaciones"]},100]},2]},
      "aprobados_en_el_anio": 1,
      "edad_promedio":
{"$cond":[{"$gt":["$n_edad",0]},{"$round":[{"$divide":["$suma_edad","$n_edad"]},2]},null]},
      "edad_promedio_aprobados": {
        "$cond": [
          {"$gt":["$n_edad_aprobados",0]},
          {"$round":[{"$divide":["$suma_edad_aprobados","$n_edad_aprobados"]},2]},
          null
        ]
      },
      "primer_anio": 1,
      "ultimo_anio": 1,
      "por_anio": {"$sortArray":{"input":"$por_anio","sortBy":{"anio_ingreso":1}}},
      "generado_en": "$$NOW"
    }
  },
  {"$merge":{"into":"resumen_carrera","on":"_id","whenMatched":"replace","whenNotMatched":"insert"}}
])
```

Resultado (220 ms · primeras 8 de 14 filas):

carrera	postulaciones	aspirantes distintos	aprobados	tasa aprobacion	edad promedio
Ing. EN CIENCIAS Y SISTEMAS	18,588	14,124	2,817	15.15	20.35
Ing. CIVIL	9,062	7,133	1,051	11.6	20.94
Ing. INDUSTRIAL	8,760	7,015	1,097	12.52	20.85
Ing. QUIMICA	4,816	3,885	902	18.73	19.92
Ing. ELECTRONICA	3,858	2,961	692	17.94	20.73
Ing. MECANICA	2,876	2,276	356	12.38	21.04
Ing. ELECTRICA	2,376	1,829	383	16.12	21.21
Ing. MECANICA INDUSTRIAL	1,965	1,610	352	17.91	20.54

Consulta 7 · Edad promedio de aprobados por carrera y tipo de institución

Pregunta. Edad promedio de los aspirantes aprobados, agrupada por carrera objetivo y tipo de institución educativa.

Cómo funciona. Usa `postulaciones` con `aprobado = true` (indicador principal) y edad válida. Después de `$unwind` sobre `carreras` se agrupa por el par (carrera, tipo de institución). Se excluyen las postulaciones sin tipo de institución.

Tabla 59. Parámetros de la consulta 7

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"aprobado":true,"edad":{"$ne":null},"tipo_institucion":{"$ne":null}}},
  {"$unwind":"$carreras"},
  {
    "$group": {
      "_id": {"carrera":"$carreras","tipo_institucion":"$tipo_institucion"},
      "edad_promedio": {"$avg":"$edad"},
      "aprobados": {"$sum":1}
    }
  },
  {"$sort":{"_id.carrera":1,"_id.tipo_institucion":1}},
  {
    "$project": {
      "_id": 0,
      "carrera": "$_id.carrera",
      "tipo_institucion": "$_id.tipo_institucion",
      "edad_promedio": {"$round":["$edad_promedio",2]},
      "aprobados": 1
    }
  }
])
```

Resultado (108 ms · primeras 8 de 36 filas):

carrera	tipo institucion	aprobados	edad promedio
Ing. AMBIENTAL	PRIVADO	106	19.36
Ing. AMBIENTAL	PUBLICO	5	19.8
Ing. CIVIL	MUNICIPAL	1	20
Ing. CIVIL	POR COOPERATIVA	5	20
Ing. CIVIL	PRIVADO	876	19.71
Ing. CIVIL	PUBLICO	103	20.7
Ing. ELECTRICA	POR COOPERATIVA	4	19.75
Ing. ELECTRICA	PRIVADO	305	19.9

Consulta 8 · Distribución de aprobados por municipio y carrera

Pregunta. Cantidad de aspirantes aprobados según el municipio de su institución y la carrera objetivo.

Cómo funciona. Usa `postulaciones` con `aprobado = true`. Se agrupa por (departamento, municipio, carrera): el departamento forma parte de la clave porque hay municipios con el mismo

nombre en departamentos distintos (R-G6). Se ordena de mayor a menor y se limita el número de filas (parámetro `limite`) porque hay cientos de combinaciones.

Tabla 60. Parámetros de la consulta 8

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`limite`	entero (1 a 2000)	200	Máximo de filas a devolver

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"aprobado":true,"departamento":{"$ne":null},"municipio":{"$ne":null}}},
  {"$unwind":"$carreras"},
  {
    "$group": {
      "_id": {"departamento":"$departamento","municipio":"$municipio","carrera":"$carreras"},
      "aprobados": {"$sum":1}
    }
  },
  {"$sort":{"aprobados":-1,"_id.departamento":1,"_id.municipio":1,"_id.carrera":1}},
  {"$limit":200},
  {
    "$project": {
      "_id": 0,
      "departamento": "$_id.departamento",
      "municipio": "$_id.municipio",
      "carrera": "$_id.carrera",
      "aprobados": 1
    }
  }
])
```

Resultado (118 ms · primeras 8 de 200 filas):

departamento	municipio	carrera	aprobados
GUATEMALA	GUATEMALA	Ing. EN CIENCIAS Y SISTEMAS	1,433
GUATEMALA	GUATEMALA	Ing. INDUSTRIAL	542
GUATEMALA	GUATEMALA	Ing. CIVIL	477
GUATEMALA	MIXCO	Ing. EN CIENCIAS Y SISTEMAS	434
GUATEMALA	GUATEMALA	Ing. ELECTRONICA	398
GUATEMALA	GUATEMALA	Ing. QUIMICA	375
GUATEMALA	VILLA NUEVA	Ing. EN CIENCIAS Y SISTEMAS	228
GUATEMALA	GUATEMALA	Ing. MECANICA	189

Consulta 9 · Evaluaciones por mes y materia (instituciones públicas)

Pregunta. Cantidad de evaluaciones realizadas cada mes, por materia, solo de aspirantes de instituciones públicas.

Cómo funciona. Usa `evaluaciones`. Se filtra `tipo_institucion = PUBLICO` y se agrupa por (año-mes, materia). El campo `anio_mes` (por ejemplo `2014-08`) se calculó en el ETL a partir de `fecha_asignacion`; agrupar por año-mes y no solo por mes del año da una serie de tiempo más informativa. Es un conteo de evaluaciones, no de personas.

Tabla 61. Parámetros de la consulta 9

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `evaluaciones`:

```
db.evaluaciones.aggregate([
  {"$match":{"tipo_institucion":"PUBLICO"}},
  {"$group":{"_id":{"anio_mes":"$anio_mes","materia":"$materia"},"evaluaciones":{"$sum":1}}},
  {"$sort":{"_id.anio_mes":1,"_id.materia":1}},
  {"$project":{"_id":0,"anio_mes":"$_id.anio_mes","materia":"$_id.materia","evaluaciones":1}}
])
```

Resultado (37 ms · primeras 8 de 99 filas):

anio mes	materia	evaluaciones
2014-08	COMPUTACION	408
2014-08	MATEMATICA	614
2014-09	COMPUTACION	206
2014-09	MATEMATICA	514
2014-10	COMPUTACION	219
2014-10	MATEMATICA	447
2014-11	COMPUTACION	29
2014-11	MATEMATICA	63

Consulta 10 · Top 5 de carreras más demandadas entre 16 y 18 años

Pregunta. Las carreras con más aspirantes dentro de un rango de edad (por defecto de 16 a 18 años).

Cómo funciona. Usa `postulaciones`. Se filtra la edad con `$gte` y `$lte` (parámetros `edad_min` y `edad_max`, por defecto 16 y 18), se despliega `carreras`, se cuenta por carrera, se ordena de mayor a menor y `$limit` deja las primeras (`limite`, por defecto 5). La demanda es el número de postulaciones (aspirante + año de ingreso). No incluye postulaciones sin carrera.

Tabla 62. Parámetros de la consulta 10

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`edad_min`	entero (13 a 70)	16	Edad mínima
`edad_max`	entero (13 a 70)	18	Edad máxima
`limite`	entero (1 a 20)	5	Cuántas carreras mostrar

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"edad":{"$gte":16,"$lte":18}}},
  {"$unwind":"$carreras"},
  {"$group":{"_id":"$carreras","aspirantes":{"$sum":1}}},
  {"$sort":{"aspirantes":-1,"_id":1}},
  {"$limit":5},
  {"$project":{"_id":0,"carrera":"$_id","aspirantes":1}}
])
```

Resultado (20 ms · 5 filas):

carrera	aspirantes
Ing. EN CIENCIAS Y SISTEMAS	3,818
Ing. CIVIL	1,374
Ing. INDUSTRIAL	1,361
Ing. QUIMICA	1,182
Ing. ELECTRONICA	409

A.2 Consultas 11 a 20

Consulta 11 · Historial de desempeño por aspirante

Pregunta. Evaluaciones, materias, intentos y resultados de un aspirante, año por año.

Cómo funciona. Se lee un solo documento de `aspirantes` (por su `_id`) y se despliega su arreglo embebido `historial`. Después de ordenar por año, oportunidad y fecha, `$group` arma una fila por (año de ingreso, materia) con el número de intentos, si aprobó (`$max` sobre booleanos: `true` es mayor que `false`) y el detalle de cada intento, que `$push` conserva en orden.

Tabla 63. Parámetros de la consulta 11

Nombre	Tipo	Por defecto	Descripción
`aspirante_id`	texto	—	Identificador del aspirante (el correlativo de 20 o 32 caracteres)

Pipeline sobre `aspirantes`:

```
db.aspirantes.aggregate([
  {"$match":{"_id":"002682dc0bebeba42770"}},
  {"$unwind":"$historial"},
  {"$sort":{"historial.anio_ingreso":1,"historial.oportunidad":1,"historial.fecha":1}},
  {
    "$group": {
      "_id": {"anio_ingreso":"$historial.anio_ingreso","materia":"$historial.materia"},
      "intentos": {"$sum":1},
      "aprobada": {"$max":"$historial.aprobada"},
      "detalle": {
        "$push": {
          "oportunidad": "$historial.oportunidad",
          "fecha": "$historial.fecha",
          "aprobada": "$historial.aprobada",
          "carrera": "$historial.carrera"
        }
      }
    }
  },
  {"$sort":{"_id.anio_ingreso":1,"_id.materia":1}},
  {
    "$project": {
      "_id": 0,
      "anio_ingreso": "$_id.anio_ingreso",
      "materia": "$_id.materia",
      "intentos": 1,
      "aprobada": 1,
      "detalle": 1
    }
  }
])
```

Resultado (2 ms · 3 filas):

materia	intentos	aprobada	anio ingreso
COMPUTACION	2	sí	2019
MATEMATICA	3	no	2019
MATEMATICA	2	sí	2020

Consulta 12 · Distribución por sexo y tipo de institución educativa

Pregunta. Cantidad de aspirantes (personas distintas) agrupados por sexo y por tipo de institución.

Cómo funciona. Usa `aspirantes` (una fila por persona). Se agrupa por el par (sexo, tipo de institución). Los aspirantes sin sexo registrado (`null` desde el ETL) se muestran como `SIN REGISTRO` con `$ifNull`; los que no tienen tipo de institución se excluyen.

Pipeline sobre `aspirantes`:

```
db.aspirantes.aggregate([
  {"$match":{"tipo_institucion":{"$ne":null}}},
  {
    "$group": {
      "_id": {"sexo":{"$ifNull":["$sexo","SIN REGISTRO"]},"tipo_institucion":"$tipo_institucion"},
      "aspirantes": {"$sum":1}
    }
  },
  {"$sort":{"_id.tipo_institucion":1,"_id.sexo":1}},
  {
    "$project": {"_id":0,"tipo_institucion":"$_id.tipo_institucion","sexo":"$_id.sexo","aspirantes":1}
  }
])
```

Resultado (43 ms · primeras 8 de 9 filas):

tipo institucion	sexo	aspirantes
MUNICIPAL	F	10
MUNICIPAL	M	45
POR COOPERATIVA	F	102
POR COOPERATIVA	M	332
PRIVADO	F	7,838
PRIVADO	M	25,633
PRIVADO	SIN REGISTRO	6
PUBLICO	F	1,213

Consulta 13 · Tasa de aprobación por edad

Pregunta. Porcentaje de aspirantes aprobados para cada edad.

Cómo funciona. Usa `postulaciones` con edad válida. Se agrupa por `edad` y se cuentan las postulaciones totales y las aprobadas (`aprobado = true`, indicador principal). La tasa es aprobados / total * 100. Las edades con muy pocos aspirantes (por encima de unos 40 años) dan porcentajes poco confiables: conviene mirar también la columna `aspirantes`.

Tabla 64. Parámetros de la consulta 13

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"edad":{"$ne":null}}},
  {
    "$group": {"_id":"$edad","aspirantes":{"$sum":1},"aprobados":{"$sum":{"$cond":["$aprobado",1,0]}}}
  },
  {"$sort":{"_id":1}},
  {
    "$project": {
      "_id": 0,
      "edad": "$_id",
      "aspirantes": 1,
      "aprobados": 1,
      "tasa_aprobacion": {"$round":[{"$multiply":[{"$divide":["$aprobados","$aspirantes"]},100]},2]}
    }
  }
])
```

Resultado (105 ms · primeras 8 de 46 filas):

aspirantes	aprobados	edad	tasa aprobacion
1	0	13	0
1	0	14	0
2	1	15	50
16	4	16	25
695	136	17	19.57
8,080	1,747	18	21.62
13,883	2,765	19	19.92
9,934	1,369	20	13.78

Consulta 14 · Número promedio de intentos por materia

Pregunta. Promedio de oportunidades de evaluación que usan los aspirantes en cada materia dentro de un año de ingreso.

Cómo funciona. Usa `postulaciones`, cuyo arreglo embebido `materias` ya trae `intentos` (cuántas evaluaciones tuvo esa materia en el año). `$unwind` da una fila por (postulación, materia) y se promedia por materia. Un "intento" es una aparición del aspirante, no el número de oportunidad: quien aparece por primera vez en la oportunidad 3 está en su intento 1.

Tabla 65. Parámetros de la consulta 14

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$unwind":"$materias"},
  {
    "$group": {
      "_id": "$materias.materia",
      "promedio_intentos": {"$avg":"$materias.intentos"},
      "maximo_intentos": {"$max":"$materias.intentos"},
      "aspirantes": {"$sum":1}
    }
  },
  {"$sort":{"aspirantes":-1}},
  {
    "$project": {
      "_id": 0,
      "materia": "$_id",
      "promedio_intentos": {"$round":["$promedio_intentos",2]},
      "maximo_intentos": 1,
      "aspirantes": 1
    }
  }
])
```

Resultado (107 ms · 3 filas):

materia	maximo intentos	aspirantes	promedio intentos
MATEMATICA	5	55,865	2.22
COMPUTACION	5	45,476	1.84
MATEMATICA ITUGS	4	14	1.57

Consulta 15 · Historial completo de un aspirante

Pregunta. Toda la información disponible de un aspirante identificado por su correlativo.

Cómo funciona. Se busca el documento de `aspirantes` por `_id` y `$lookup` trae todas sus `postulaciones` (la colección derivada, con el estado de aprobación de cada año). `$sortArray` ordena las postulaciones por año. El resultado reúne en un solo documento los datos de la persona, su habilitación, cada postulación y el historial completo de evaluaciones.

Tabla 66. Parámetros de la consulta 15

Nombre	Tipo	Por defecto	Descripción
`aspirante_id`	texto	—	Identificador del aspirante (el correlativo de 20 o 32 caracteres)

Pipeline sobre `aspirantes`:

```
db.aspirantes.aggregate([
  {"$match":{"_id":"002682dc0bebeba42770"}},
  {
    "$lookup": {"from":"postulaciones","localField":"_id","foreignField":"aspirante_id","as":"postulaciones"}
  },
  {
    "$project": {
      "correlativo_original": 1,
      "sexo": 1,
      "anio_nacimiento": 1,
      "tipo_institucion": 1,
      "departamento": 1,
      "municipio": 1,
      "anios_ingreso": 1,
      "habilitacion": 1,
      "advertencias": 1,
      "postulaciones": {"$sortArray":{"input":"$postulaciones","sortBy":{"anio_ingreso":1}}},
      "historial": 1
    }
  }
])
```

Resultado (2 ms · 1 fila):

id	correlativo original	sexo	tipo institucion	departamento	municipio
002682dc0bebeba42770	002682dc0bebeba427703d33aae5cb14	M	PRIVADO	GUATEMALA	GUATEMALA

Consulta 16 · Carreras con más aspirantes reprobados en su primer intento

Pregunta. Carreras objetivo con mayor cantidad de aspirantes que no aprobaron en su primera aparición del año.

Cómo funciona. El primer intento de un aspirante son las evaluaciones de su MENOR oportunidad en el año (no tiene que ser la 1; R-P1), que el ETL guardó en `primer_intento`. Con `criterio=alguna` (por defecto) cuenta como reprobado quien falló AL MENOS UNA materia en ese primer intento; con `criterio=todas` solo quien falló todas (`$anyElementTrue` sobre los resultados). Se despliegan las carreras del primer intento y se compara contra el total de aspirantes de esa carrera.

Tabla 67. Parámetros de la consulta 16

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`criterio`	opcion	alguna	alguna = reprobó al menos una materia; todas = reprobó todas las que presentó

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$unwind":"$primer_intento.carreras"},
  {
    "$group": {
      "_id": "$primer_intento.carreras",
      "aspirantes": {"$sum":1},
      "reprobados": {"$sum":{"$cond":["$primer_intento.reprobo",1,0]}}
    }
  },
  {"$sort":{"reprobados":-1,"_id":1}},
  {
    "$project": {
      "_id": 0,
      "carrera": "$_id",
      "aspirantes": 1,
      "reprobados": 1,
      "porcentaje_reprobados": {"$round":[{"$multiply":[{"$divide":["$reprobados","$aspirantes"]},100]},2]}
    }
  }
])
```

Resultado (90 ms · primeras 8 de 14 filas):

carrera	aspirantes	reprobados	porcentaje reprobados
Ing. EN CIENCIAS Y SISTEMAS	17,908	17,431	97.34
Ing. CIVIL	8,175	7,998	97.83
Ing. INDUSTRIAL	7,765	7,593	97.78
Ing. QUIMICA	4,544	4,377	96.32
Ing. ELECTRONICA	3,424	3,295	96.23
Ing. MECANICA	2,527	2,476	97.98

carrera	aspirantes	reprobados	porcentaje reprobados
Ing. ELECTRICA	2,024	1,957	96.69
Ing. MECANICA INDUSTRIAL	1,580	1,533	97.03

Consulta 17 · Top 5 de municipios con más aspirantes

Pregunta. Los cinco municipios de procedencia de las instituciones con mayor cantidad de aspirantes.

Cómo funciona. Usa `aspirantes` (cada persona cuenta una vez). La clave de agrupación es (departamento, municipio), porque hay municipios con el mismo nombre en departamentos distintos (R-G6). La capital aparece como un solo municipio, `GUATEMALA`, sumando todas sus zonas (R-G4). `$concat` arma una etiqueta legible para la gráfica.

Tabla 68. Parámetros de la consulta 17

Nombre	Tipo	Por defecto	Descripción
`limite`	entero (1 a 50)	5	Cuántos municipios mostrar

Pipeline sobre `aspirantes`:

```
db.aspirantes.aggregate([
  {"$match":{"departamento":{"$ne":null},"municipio":{"$ne":null}}},
  {
    "$group": {"_id":{"departamento":"$departamento","municipio":"$municipio"},"aspirantes":{"$sum":1}}
  },
  {"$sort":{"aspirantes":-1,"_id.departamento":1,"_id.municipio":1}},
  {"$limit":5},
  {
    "$project": {
      "_id": 0,
      "departamento": "$_id.departamento",
      "municipio": "$_id.municipio",
      "lugar": {"$concat":["$_id.municipio"," (","$_id.departamento",")"]},
      "aspirantes": 1
    }
  }
])
```

Resultado (54 ms · 5 filas):

departamento	municipio	lugar	aspirantes
GUATEMALA	GUATEMALA	GUATEMALA (GUATEMALA)	19,024
GUATEMALA	MIXCO	MIXCO (GUATEMALA)	4,905
GUATEMALA	VILLA NUEVA	VILLA NUEVA (GUATEMALA)	3,134
CHIMALTENANGO	CHIMALTENANGO	CHIMALTENANGO (CHIMALTENANGO)	1,399
GUATEMALA	PETAPA	PETAPA (GUATEMALA)	938

Consulta 18 · Tasa de aprobación por tipo de institución educativa

Pregunta. Porcentaje de aspirantes aprobados en cada tipo de institución, para compararlos.

Cómo funciona. Usa `postulaciones`. Se agrupa por `tipo_institucion` y se cuentan las postulaciones totales y las aprobadas (indicador principal). La tasa es aprobados / total * 100. Se excluyen las postulaciones sin tipo de institución.

Tabla 69. Parámetros de la consulta 18

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"tipo_institucion":{"$ne":null}}},
  {
    "$group": {
      "_id": "$tipo_institucion",
      "aspirantes": {"$sum":1},
      "aprobados": {"$sum":{"$cond":["$aprobado",1,0]}}
    }
  },
  {"$sort":{"aspirantes":-1}},
  {
    "$project": {
      "_id": 0,
      "tipo_institucion": "$_id",
      "aspirantes": 1,
      "aprobados": 1,
      "tasa_aprobacion": {"$round":[{"$multiply":[{"$divide":["$aprobados","$aspirantes"]},100]},2]}
    }
  }
])
```

Resultado (68 ms · 4 filas):

tipo institucion	aspirantes	aprobados	tasa aprobacion
PRIVADO	46,369	7,513	16.2
PUBLICO	8,055	845	10.49
POR COOPERATIVA	637	65	10.2
MUNICIPAL	77	4	5.19

Consulta 19 · Carreras más demandadas por departamento

Pregunta. Las carreras con mayor demanda dentro de cada departamento de procedencia de la institución.

Cómo funciona. Usa `postulaciones`; la demanda es el número de postulaciones por carrera. Es un "top N por grupo": primero se cuenta por (departamento, carrera) y se ordena; luego un segundo `$group` junta las carreras de cada departamento en un arreglo ya ordenado (`$push`), `$slice` se queda con las primeras N (parámetro `top`) y `$unwind` con `includeArrayIndex` vuelve a convertirlas en filas numeradas (la posición en el ranking).

Tabla 70. Parámetros de la consulta 19

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`top`	entero (1 a 14)	3	Cuántas carreras mostrar por departamento

Pipeline sobre `postulaciones`:

```
db.postulaciones.aggregate([
  {"$match":{"departamento":{"$ne":null}}},
  {"$unwind":"$carreras"},
  {"$group":{"_id":{"departamento":"$departamento","carrera":"$carreras"},"demanda":{"$sum":1}}},
  {"$sort":{"_id.departamento":1,"demanda":-1,"_id.carrera":1}},
  {
    "$group": {"_id":"$_id.departamento","carreras":{"$push":{"carrera":"$_id.carrera","demanda":"$demanda"}}}
  },
  {"$project":{"top":{"$slice":["$carreras",3]}}},
  {"$unwind":{"path":"$top","includeArrayIndex":"indice"}},
  {"$sort":{"_id":1,"indice":1}},
  {
    "$project": {
      "_id": 0,
      "departamento": "$_id",
      "posicion": {"$add":["$indice",1]},
      "carrera": "$top.carrera",
      "demanda": "$top.demanda"
    }
  }
])
```

Resultado (140 ms · primeras 8 de 69 filas):

departamento	carrera	posicion	demanda
ALTA VERAPAZ	Ing. CIVIL	1	258
ALTA VERAPAZ	Ing. EN CIENCIAS Y SISTEMAS	2	166
ALTA VERAPAZ	Ing. INDUSTRIAL	3	138
BAJA VERAPAZ	Ing. EN CIENCIAS Y SISTEMAS	1	46
BAJA VERAPAZ	Ing. CIVIL	2	44
BAJA VERAPAZ	Ing. INDUSTRIAL	3	24
CHIMALTENANGO	Ing. EN CIENCIAS Y SISTEMAS	1	782
CHIMALTENANGO	Ing. CIVIL	2	416

Consulta 20 · Evolución de la aprobación por año de ingreso

Pregunta. Cantidad y porcentaje de evaluaciones aprobadas y no aprobadas en cada año de ingreso.

Cómo funciona. Nivel EVALUACIÓN. Se agrupa `evaluaciones` por `anio_ingreso` contando el total y las aprobadas; las no aprobadas son la diferencia (`$subtract`) y los porcentajes se calculan sobre el total del año. Como cada intento fallido cuenta, los porcentajes son bajos aunque muchos aspirantes terminen aprobando: la consulta 4 y el indicador de aspirantes dan otra mirada.

Pipeline sobre `evaluaciones`:

```
db.evaluaciones.aggregate([
  {
    "$group": {
      "_id": "$anio_ingreso",
      "evaluaciones": {"$sum":1},
      "aprobadas": {"$sum":{"$cond":["$aprobada",1,0]}}
    }
  },
  {"$sort":{"_id":1}},
  {
    "$project": {
      "_id": 0,
      "anio_ingreso": "$_id",
      "evaluaciones": 1,
      "aprobadas": 1,
      "no_aprobadas": {"$subtract":["$evaluaciones","$aprobadas"]},
      "porcentaje_aprobadas": {"$round":[{"$multiply":[{"$divide":["$aprobadas","$evaluaciones"]},100]},2]},
      "porcentaje_no_aprobadas": {
        "$round": [
          {
            "$multiply": [{"$divide":[{"$subtract":["$evaluaciones","$aprobadas"]},"$evaluaciones"]},100]
          },
          2
        ]
      }
    }
  }
])
```

Resultado (202 ms · primeras 8 de 10 filas):

evaluaciones	aprobadas	anio ingreso	no aprobadas	porcentaje aprobadas	porcentaje no aprobadas
20,266	4,743	2015	15,523	23.4	76.6
21,987	3,084	2016	18,903	14.03	85.97
21,398	3,328	2017	18,070	15.55	84.45
20,678	3,845	2018	16,833	18.59	81.41
17,835	3,637	2019	14,198	20.39	79.61
17,665	3,334	2020	14,331	18.87	81.13
30,642	1,681	2021	28,961	5.49	94.51
24,231	1,661	2022	22,570	6.85	93.15

Anexo B. Consultas de Cypher

Las 6 consultas de Neo4j, con el Cypher exacto que ejecuta el servidor y las primeras filas del resultado. Los parámetros comienzan con `$`; en Neo4j Browser se pueden fijar con `:param nombre => valor` antes de ejecutar la consulta.

B.1 Consultas G1 a G6

Consulta G1 · Carreras más demandadas desde un departamento

Pregunta. Qué carreras eligen los aspirantes que proceden de un departamento, opcionalmente en un año de ingreso.

Cómo funciona. Recorre tres saltos del grafo: Carrera ← Postulacion → Municipio → Departamento. Se parte del nodo `Departamento` elegido, se sube por `PERTENECE_A` hasta sus `Municipio`, se baja por `PROCEDE_DE` hasta las `Postulacion` y se sigue `ASPIRA_A` hasta la `Carrera`. El filtro por año usa la propiedad `anio_ingreso` de la postulación; `$anio_ingreso IS NULL` hace que el filtro sea opcional.

Qué ilustra del lenguaje:

- Recorrido de 3 relaciones
- Filtro por propiedad de nodo
- Agregación con count

Tabla 71. Parámetros de la consulta 1

Nombre	Tipo	Por defecto	Descripción
`departamento`	texto	QUETZALTENANGO	Departamento de la institución (en mayúsculas, por ejemplo GUATEMALA)
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`limite`	entero (1 a 30)	10	Cuántas carreras mostrar

Consulta:

```
MATCH (d:Departamento {nombre:
$departamento})<-[:PERTENECE_A]-(:Municipio)<-[:PROCEDE_DE]-(p:Postulacion)-[:ASPIRA_A]->(c:Carrera)
WHERE $anio_ingreso IS NULL OR p.anio_ingreso = $anio_ingreso
RETURN c.nombre AS carrera,
       count(p) AS postulaciones,
       sum(CASE WHEN p.aprobado THEN 1 ELSE 0 END) AS aprobados
ORDER BY postulaciones DESC
LIMIT $limite
```

Resultado (107 ms · primeras 8 de 10 filas):

carrera	postulaciones	aprobados
Ing. EN CIENCIAS Y SISTEMAS	162	19
Ing. CIVIL	124	15
Ing. INDUSTRIAL	68	7
Ing. QUIMICA	65	20
Ing. ELECTRICA	46	8

carrera	postulaciones	aprobados
Ing. ELECTRONICA	36	4
Ing. MECANICA ELECTRICA	34	9
Ing. MECANICA	29	4

Consulta G2 · Pares de carreras que comparten aspirantes

Pregunta. Qué carreras aparecen juntas en una misma postulación: aspirantes que dudan entre dos opciones.

Cómo funciona. Un patrón con dos relaciones `ASPIRA_A` que salen de la misma `Postulacion` hacia dos `Carrera` distintas. La condición `c1.nombre < c2.nombre` evita contar cada par dos veces (A–B y B–A). Es una consulta muy natural en grafos: dos nodos que se conectan a través de un tercero. En MongoDB habría que comparar arreglos de carreras.

Qué ilustra del lenguaje:

- Patrón con un nodo intermedio compartido
- Dos relaciones del mismo tipo
- Filtro opcional por propiedad

Tabla 72. Parámetros de la consulta 2

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`limite`	entero (1 a 50)	10	Cuántos pares mostrar

Consulta:

```
MATCH (c1:Carrera)<-[:ASPIRA_A]-(p:Postulacion)-[:ASPIRA_A]->(c2:Carrera)
WHERE c1.nombre < c2.nombre
  AND ($anio_ingreso IS NULL OR p.anio_ingreso = $anio_ingreso)
RETURN c1.nombre AS carrera_a, c2.nombre AS carrera_b, count(p) AS postulaciones
ORDER BY postulaciones DESC
LIMIT $limite
```

Resultado (97 ms · primeras 8 de 10 filas):

carrera a	carrera b	postulaciones
Ing. CIVIL	Ing. EN CIENCIAS Y SISTEMAS	892
Ing. CIVIL	Ing. INDUSTRIAL	654
Ing. EN CIENCIAS Y SISTEMAS	Ing. INDUSTRIAL	446
Ing. ELECTRONICA	Ing. EN CIENCIAS Y SISTEMAS	354
Ing. CIVIL	Ing. QUIMICA	304
Ing. INDUSTRIAL	Ing. MECANICA INDUSTRIAL	297
Ing. ELECTRICA	Ing. ELECTRONICA	236
Ing. MECANICA	Ing. MECANICA INDUSTRIAL	209

Consulta G3 · Aspirantes que cambian de carrera de un año al siguiente

Pregunta. De qué carrera a qué carrera se mueven los aspirantes que vuelven a postular al año siguiente.

Cómo funciona. El recorrido cruza dos años a través del nodo `Aspirante`: Carrera ← Postulacion (año N) ← Aspirante → Postulacion (año N+1) → Carrera. La condición `p2.anio_ingreso = p1.anio_ingreso + 1` limita a años consecutivos. Las dos negaciones `NOT (p)-[:ASPIRA_A]->(c)` descartan a quien conserva una de sus carreras, de modo que solo cuentan cambios reales. El parámetro `anio_ingreso`, si se indica, es el año al que llegan (N+1).

Qué ilustra del lenguaje:

- Recorrido que cruza años a través del aspirante
- Aritmética sobre propiedades
- Negación de patrones

Tabla 73. Parámetros de la consulta 3

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Año al que llegan los aspirantes (el año siguiente al de origen)
`limite`	entero (1 a 50)	15	Cuántos cambios mostrar

Consulta:

```
MATCH (a:Aspirante)-[:POSTULO]->(p1:Postulacion)-[:ASPIRA_A]->(c1:Carrera),
      (a)-[:POSTULO]->(p2:Postulacion)-[:ASPIRA_A]->(c2:Carrera)
WHERE p2.anio_ingreso = p1.anio_ingreso + 1
  AND NOT (p2)-[:ASPIRA_A]->(c1)
  AND NOT (p1)-[:ASPIRA_A]->(c2)
  AND ($anio_ingreso IS NULL OR p2.anio_ingreso = $anio_ingreso)
RETURN c1.nombre AS desde, c2.nombre AS hacia, count(DISTINCT a) AS aspirantes
ORDER BY aspirantes DESC
LIMIT $limite
```

Resultado (665 ms · primeras 8 de 15 filas):

desde	hacia	aspirantes
Ing. EN CIENCIAS Y SISTEMAS	Ing. INDUSTRIAL	142
Ing. CIVIL	Ing. INDUSTRIAL	102
Ing. INDUSTRIAL	Ing. EN CIENCIAS Y SISTEMAS	81
Ing. ELECTRONICA	Ing. EN CIENCIAS Y SISTEMAS	72
Ing. EN CIENCIAS Y SISTEMAS	Ing. ELECTRONICA	69
Ing. CIVIL	Ing. EN CIENCIAS Y SISTEMAS	69
Ing. EN CIENCIAS Y SISTEMAS	Ing. CIVIL	54
Ing. INDUSTRIAL	Ing. CIVIL	51

Consulta G4 · Aprobaron Matemática tras varios intentos, por tipo de institución

Pregunta. Entre quienes aprobaron Matemática, cuántos lo lograron hasta su intento N o posterior, según el tipo de institución.

Cómo funciona. Filtra por una propiedad de la RELACIÓN: `EVALUADA_EN` guarda `aprobada` e `intento` (la posición de la evaluación entre las de esa materia en el año). Se parte de la `Materia` Matemática, se toman las evaluaciones aprobadas y se une cada postulación con su `TipoInstitucion`. `count(DISTINCT CASE ... END)` cuenta solo las postulaciones cuyo intento aprobado fue ≥ `intento_minimo`. Un aspirante con dos aprobaciones se cuenta una sola vez gracias a `DISTINCT`.

Qué ilustra del lenguaje:

- Propiedades en relaciones
- Filtro sobre propiedades de la relación
- Unión con un segundo recorrido

Tabla 74. Parámetros de la consulta 4

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`intento_minimo`	entero (1 a 10)	3	Intento a partir del cual se considera "tras varios intentos"

Consulta:

```
MATCH (p:Postulacion)-[r:EVALUADA_EN]->(:Materia {nombre: 'MATEMATICA'})
WHERE r.aprobada
  AND ($anio_ingreso IS NULL OR p.anio_ingreso = $anio_ingreso)
MATCH (p)-[:EN_INSTITUCION]->(t:TipoInstitucion)
WITH t.nombre AS tipo_institucion,
     count(DISTINCT p) AS aprobaron,
     count(DISTINCT CASE WHEN r.intento >= $intento_minimo THEN p END) AS tras_varios_intentos
RETURN tipo_institucion, aprobaron, tras_varios_intentos,
       round(100.0 * tras_varios_intentos / aprobaron, 2) AS porcentaje
ORDER BY aprobaron DESC
```

Resultado (319 ms · 4 filas):

tipo institucion	aprobaron	tras varios intentos	porcentaje
PRIVADO	9,631	3,593	37.31
PUBLICO	1,114	445	39.95
POR COOPERATIVA	80	36	45
MUNICIPAL	6	6	100

Consulta G5 · Flujo de aprobados: de qué departamentos a qué carreras

Pregunta. Combinaciones departamento → carrera con más aspirantes aprobados (habilitados).

Cómo funciona. Un camino largo del grafo: Departamento ← Municipio ← Postulacion → Carrera, pero restringido a las postulaciones con la propiedad `aprobado: true`, que se escribe directamente dentro del patrón del nodo. Cada fila es una "arista" departamento–carrera ponderada por el número de aprobados: es la misma información que dibuja el mapa de demanda del explorador de grafos.

Qué ilustra del lenguaje:

- Camino de varias relaciones
- Propiedad de nodo dentro del patrón
- Resultado como aristas ponderadas

Tabla 75. Parámetros de la consulta 5

Nombre	Tipo	Por defecto	Descripción
`anio_ingreso`	entero (2000 a 2100)	—	Limita la consulta a un año de ingreso
`limite`	entero (1 a 100)	20	Cuántos flujos mostrar

Consulta:

```
MATCH (d:Departamento)<-[:PERTENECE_A]-(:Municipio)<-[:PROCEDE_DE]-(p:Postulacion {aprobado:
true})-[:ASPIRA_A]->(c:Carrera)
WHERE $anio_ingreso IS NULL OR p.anio_ingreso = $anio_ingreso
RETURN d.nombre AS departamento, c.nombre AS carrera, count(p) AS aprobados
ORDER BY aprobados DESC, departamento, carrera
LIMIT $limite
```

Resultado (224 ms · primeras 8 de 20 filas):

departamento	carrera	aprobados
GUATEMALA	Ing. EN CIENCIAS Y SISTEMAS	2,338
GUATEMALA	Ing. INDUSTRIAL	911
GUATEMALA	Ing. CIVIL	760
GUATEMALA	Ing. QUIMICA	674
GUATEMALA	Ing. ELECTRONICA	554
GUATEMALA	Ing. MECANICA	285
GUATEMALA	Ing. MECANICA INDUSTRIAL	284
GUATEMALA	Ing. ELECTRICA	271

Consulta G6 · Aspirantes que postulan varios años y cuántos terminan aprobando

Pregunta. Para quienes participaron en N años o más, cuántos llegaron a aprobar en alguno de ellos.

Cómo funciona. Cuenta las relaciones `POSTULO` de cada `Aspirante` (cuántos años participó) y, con `max(CASE ...)`, si alguna de sus postulaciones quedó con `aprobado = true`. El primer `WITH` agrupa por aspirante; el `RETURN` vuelve a agrupar por número de años. Permite comparar el porcentaje de quienes terminan aprobando según cuántos años participaron.

Qué ilustra del lenguaje:

- Conteo de relaciones por nodo
- Agregación en dos niveles con WITH
- Comparación entre grupos

Tabla 76. Parámetros de la consulta 6

Nombre	Tipo	Por defecto	Descripción
`minimo_anios`	entero (1 a 10)	1	Mínimo de años de participación

Consulta:

```
MATCH (a:Aspirante)-[:POSTULO]->(p:Postulacion)
WITH a, count(p) AS anios, max(CASE WHEN p.aprobado THEN 1 ELSE 0 END) AS aprobo
WHERE anios >= $minimo_anios
RETURN anios AS anios_de_participacion,
       count(a) AS aspirantes,
       sum(aprobo) AS aprobaron,
       round(100.0 * sum(aprobo) / count(a), 2) AS porcentaje_aprobaron
ORDER BY anios_de_participacion
```

Resultado (228 ms · primeras 8 de 10 filas):

anios de participacion	aspirantes	aprobaron	porcentaje aprobaron
1	28,229	6,456	22.87
2	9,105	1,791	19.67
3	2,016	200	9.92
4	548	19	3.47
5	192	5	2.6
6	69	0	0
7	28	0	0
8	5	0	0

B.2 Vistas del explorador de grafos

El explorador de grafos no ejecuta una de las 6 consultas anteriores, sino dos consultas especializadas que devuelven un subgrafo acotado (sección 6.6).

Vista A · Mapa de demanda

Departamento – Municipio – Carrera. El peso de cada conexión es el número de postulaciones. Sin filtros (límite de 150 conexiones) devuelve 69 nodos y 188 aristas.

```
MATCH (d:Departamento)<-[:PERTENECE_A]-(m:Municipio)<-[:PROCEDE_DE]-(p:Postulacion)-[:ASPIRA_A]->(c:Carrera)
WHERE ($anio_ingreso IS NULL OR p.anio_ingreso = $anio_ingreso)
  AND ($departamento IS NULL OR d.nombre = $departamento)
  AND (NOT $solo_aprobados OR p.aprobado)
RETURN d.nombre AS departamento, m.id AS municipio_id, m.nombre AS municipio, c.nombre AS carrera, count(p) AS
peso
ORDER BY peso DESC
LIMIT $limite
```

Vista B · Red de un aspirante

El aspirante, sus postulaciones, carreras, lugar, institución y una arista por evaluación. Para el aspirante de ejemplo devuelve 8 nodos y 15 aristas.

```
MATCH (a:Aspirante {id: $id})-[po:POSTULO]->(p:Postulacion)
OPTIONAL MATCH (p)-[r:ASPIRA_A|PROCEDE_DE|EN_INSTITUCION|EVALUADA_EN]->(x)
RETURN a, po, p, r, x
```